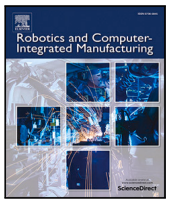
Robotics and Computer–Integrated Manufacturing 98 (2026) 103107



Contents lists available at ScienceDirect

Robotics and Computer-Integrated Manufacturing

journal homepage: www.elsevier.com/locate/rcim



Full length Article



Long short term robot safe control framework based on hidden action space heuristic soft actor–critic

Yixuan Liang, Ze Wang, Yunan Wang, Jichuan Yu, Jizhou Yan, Shize Lin, Zhao Jin, Jiuru Lu, Chuxiong Hu *

State Key Laboratory of Tribology in Advanced Equipment, Department of Mechanical Engineering, Tsinghua University, Beijing, 100084, China
Beijing Key Laboratory of Transformative High-end Manufacturing Equipment and Technology, Department of Mechanical Engineering, Tsinghua University, Beijing 100084, China

ARTICLE INFO

Keywords:
Safe control
Motion planning
Reinforcement learning
Convex optimization
Robot control system

ABSTRACT

Real-time safety control for robots in dynamic environments is a critical and challenging problem in robotics. With the advent of intelligent manufacturing, the demand for advanced safety control technologies in robotics has steadily increased. Robot planning typically involves global and local approaches, but both face limitations in real-time safety control in dynamic environments with unknown obstacles. Recent hybrid frameworks have shown progress, but challenges persist, including limited perception capabilities and poor coordination between global and local components. To address these challenges, this work proposes a novel long short term safety control framework leveraging reinforcement learning for decision-making. Perception and planning are decoupled into long-term and short-term components, with long-term perception utilizing unsupervised clustering DBSCAN for structured environment information and short-term perception enhancing efficiency through prior knowledge. Long-term planning provides reference trajectories based on static environments, while short-term planning adjusts these trajectories in real time for local safety using control barrier functions. Based on hidden action heuristic soft actor–critic and curriculum learning, the decision-making mechanism ensures safety during obstacles or attacks and maximizes robot efficiency without compromising safety. Experiments are conducted with 10,000 randomized obstacle collision scenarios, and our framework is compared with four methods, including SAC and manually designed trajectory adjustment. The results demonstrate that our approach outperforms these methods in both safety performance and operational efficiency. Finally, the system is successfully implemented in a physical environment, showcasing its practical potential for real-world applications.

1. Introduction

In modern robotics, safe control of autonomous systems in complex and unknown environments is one of the most challenging issues [1]. However, in today's manufacturing industry, robotic arms increasingly replace human operators in high-risk and repetitive inspection tasks due to their flexibility, stability, and precision [2]. Meanwhile, with the continuous advancement of manufacturing technologies and the advent of Industry 4.0, the operational demands on robots have become more flexible. New paradigms in industrial robotics no longer require strict physical separation between robots and human workers. Furthermore, to optimize production, humans and robots are expected to collaborate [3], introducing various uncertainties and dynamic events into the environment. These developments in manufacturing settings highlight the importance of ensuring that robots can operate safely and effectively in dynamic and uncertain environments.

To enable robots to avoid obstacles, two types of trajectory planning methods are commonly employed: global planning [4] and local planning [5]. For global planning, the mainstream approaches are categorized into two types. The first type consists of optimization-based trajectory planning methods [6]. These methods aim to express the dynamics of robotic arms as dynamic constraints, while safety requirements during planning are modeled as path constraints. By solving an optimization problem, the trajectory for the robot can be determined. The second type is sampling-based methods [7], which generate feasible trajectories by performing random sampling in the robot's configuration space and verifying the safety of sampled points.

Optimization-based methods have been widely applied in robotic arm control. These approaches formulate trajectory planning as a constrained optimization problem, allowing direct consideration of system dynamics, environmental constraints, and task requirements to

* Corresponding author.
E-mail address: cxhu@tsinghua.edu.cn (C. Hu).

https://doi.org/10.1016/j.rcim.2025.103107
Received 16 December 2024; Received in revised form 17 April 2025; Accepted 7 August 2025
Available online 8 September 2025
0736-5845/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

generate efficient and smooth trajectories. Notable algorithms in this category include CHOMP [8], TrajOpt [9], STOMP [10], and Model Predictive Control with constraints [11,12]. A prominent advantage of employing optimization methods is the ability to achieve time-optimal control. For example, Wang et al. demonstrated time-optimal control for high-order chain-of-integrators systems with full state constraints and arbitrary terminal states [13]. Similarly, Lin et al. proposed a local greedy search approach for multi-axis globally time-optimal trajectories [14]. However, optimization-based methods face challenges in real-world applications. Their computation time is often uncertain, and the computational complexity multiplies with the number of planning steps, making it difficult to achieve real-time performance in complex environments [15]. Additionally, since real-world problems are typically non-convex, these methods are sensitive to initial values, potentially leading to local minima and compromising the quality of the planned trajectory [16]. As a result, the ability of these methods to respond effectively to dynamic changes and complex constraints is limited, which affects their performance in real-time applications.

Sampling-based methods, such as the A* algorithm [17], probabilistic roadmap (PRM) [18], and the rapidly-exploring random tree (RRT) family of algorithms [19–26], are widely applied in robotic arm path planning. Among these, the RRT algorithm [16] and its variants are particularly popular. RRT achieves safe control by random sampling in the configuration space and connecting samples to form a feasible and collision-free path. To address the lack of smoothness and time efficiency in RRT, Kuffner and LaValle proposed the RRT-Connect algorithm [20], which adopts a bidirectional extension strategy. This approach simultaneously grows two trees from the start and goal points, attempting to connect them, significantly improving search efficiency and computational speed. However, these algorithms still face challenges, such as poor real-time performance and high computational cost [26]. As environmental complexity and the degrees of freedom (DOF) of robotic arms increase, the real-time performance of these methods becomes increasingly constrained.

Local planning methods achieve safe control by encoding obstacle information within the robot's local environment. Common techniques include the artificial potential field (APF) method [27–29] and control barrier functions (CBF) [30,31]. The Artificial Potential Field (APF) method is a classical local planning strategy that guides the robot by constructing a virtual force field. Classically, the robot is attracted to the goal by an attractive potential U_{att}, while being repelled from obstacles via a repulsive potential U_{rep}, resulting in a combined potential $U(\mathbf{q}) = U_{\text{att}}(\mathbf{q}) + U_{\text{rep}}(\mathbf{q})$. The control action is derived from the artificial force $\mathbf{F}(\mathbf{q}) = -\nabla U(\mathbf{q})$. Recent studies have demonstrated the capability and effectiveness of manually constructed potential fields in achieving real-time obstacle avoidance [32–34]. However, the APF method suffers from local minima, which can trap the robot [2].

Regarding CBF-based methods, the Control Barrier Function (CBF) framework provides a formal method for enforcing safety constraints in robotic systems. Given a safety set defined by a continuously differentiable function $h(\mathbf{x}) \geq 0$, where $\mathbf{x}$ is the system state, a valid control input must ensure that the system remains within this set. This is typically achieved by enforcing the condition $\dot{h}(\mathbf{x}) + \alpha(h(\mathbf{x})) \geq 0$, where $\alpha(\cdot)$ is an extended class-$\mathcal{K}$ function. CBFs can be incorporated into optimization-based controllers, such as quadratic programs (QPs), to ensure forward invariance of the safe set. As a result, CBFs offer a powerful tool for real-time safety control, enabling robotic systems to operate safely in dynamic and uncertain environments while respecting system dynamics and constraints. However, only a few studies have explored the integration of real-world perception inputs into safe controllers with CBF. Most research assumes that the CBF is already known and that obstacles have predefined shapes [35], which may be difficult for practical applications.

In recent years, with the increasing requirements from the industry, many studies have begun to integrate global and local planning to achieve safe control for robots. He et al. proposed a hierarchical safety control framework that decomposes safety control into two levels of optimization problems, achieving promising results in simulation [36]. Yu et al. introduced the HLSTS framework for the safety control of multiple robotic arms by considering global and local factors, demonstrating its safety in simulation [37]. Vidyasagar et al. improved the RRT-Connect algorithm by defining a new cost function, enabling the algorithm to perform local planning in regions with high-cost functions (e.g., obstacles), showing significant improvements over standard methods in simulation [38]. However, many existing studies still need to be expanded to simulation environments, where obstacles are often predefined to regular objects. Furthermore, existing studies usually need more intelligent and efficient mechanisms to coordinate the relationship between local and global planning, limiting their ability to accomplish global tasks.

Some efforts have attempted to achieve safe control through safety-aware time(trajectory) scaling methods. Matteo et al. modeled obstacles as articulated linkage models to perform safety modeling. They applied trajectory scaling to ensure safe control in Human-Robot Collaboration (HRC) scenarios, enabling the robot to scale down its trajectory or stop in hazardous situations [3]. Lucas et al. proposed an optimization-based joint velocity controller that dynamically adjusts joint velocity limits based on the robot's braking capacity and the distance between the robot and the human [39]. Similarly, Andrea et al. employed the ISO/TS 15066 standard to adjust the robot's speed in real-time based on local information to ensure operator safety [40]. However, these approaches mainly rely on passive safety strategies, such as slowing down and barking, which may struggle to handle diverse and complex scenarios.

With the rise of Deep Reinforcement Learning (DRL), some studies have explored DRL-based approaches for safe control. For example, Thumn et al. utilized Soft Actor–Critic (SAC) to evaluate the current safety state and scaled the robot's global trajectory based on local safety assessments [41]. Hsu et al. developed a novel buffer system for local hazards, clustering experiences using k-means to provide critical references for DRL in hazardous states. This method enhanced the safety of DRL and achieved notable improvements in simulation experiments [42]. Li et al. proposed a safe and robust deep reinforcement learning algorithm for constrained zero-sum scenarios by incorporating two adversarial networks [43]. However, most of these studies are still limited to simulation environments.

To address the aforementioned challenges, including environmental perception, the integration of global and local planning, their intelligent coordination, and practical applicability in real-world physical environments, a systematic approach to safety control is essential to resolve these issues effectively. This study proposes a DRL-based framework for long short term robot safe control, integrating perception, decision-making, planning, and control. As shown in Fig. 1, the framework of our study represents a comprehensive and systematic approach to robot safety control.

The key contributions of this study are described as follows:

1. Efficient Long-Short Term Perception: A novel perception method that decouples global and local perception is proposed in this study. By employing unsupervised learning method DBSCAN for long-term perception and using it as prior information to enhance short-term perception, The inefficiencies of traditional perception methods are addressed in this study, achieving a more efficient perception mechanism.
2. HSAC-based Coordination Mechanism: A Hidden Action Heuristic Soft Actor–Critic (HSAC) method is introduced in this study, which proactively predicts risks and adjusts the robot's motion accordingly. This mechanism ensures safety in the presence of obstacles or active attacks while avoiding the efficiency loss caused by overly conservative strategies seen in traditional methods.

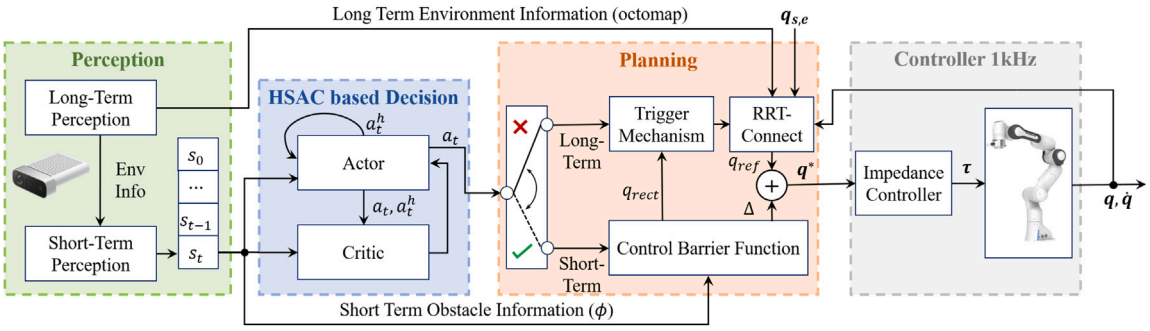


Fig. 1. Overview of the proposed framework: Integration of long short term perception, decision-making, planning and control for safe robot control.

3. Systematic Safety Control Framework: A hierarchical robot safety control framework based on HSAC is presented, incorporating perception, decision-making, planning, and low-level control. This framework outperforms traditional methods in both safety and operational efficiency, and its effectiveness is validated through both simulation and real-world experiments, demonstrating its practical applicability.

2. Framework overview

Specifically, the Overview of the proposed framework is illustrated in Fig. 1. The process begins with the perception component, divided into long-term and short-term perceptions. The long-term perception component focuses on global environmental information. It utilizes RGB-D sensors to capture point clouds of the current space and processes this data through filtering, denoising, and detection methods to obtain environmental information. This information is then parameterized into features that facilitate long-term planning. Conversely, short-term perception focuses solely on the most immediate threats to system safety, parameterizing this information to produce a safety indicator function that guides the robot in making short-term trajectory corrections. Perception information will also be sent to the decision-making module, which coordinates long-term and short-term planning relationships. Based on the DRL algorithm HSAC, this module determines when the control system should only adjust the reference trajectory and when it is necessary to invest additional time to plan a new trajectory that better ensures safety and efficiency, thereby avoiding local optima. Once a decision is made, robot planning is initiated immediately. Robot planning is divided into two levels: long-term and short-term. Long-term planning utilizes the parameterized environmental information to plan a long-term safety reference trajectory. Short-term planning then adjusts the reference trajectory based on real-time information from short-term perception. Furthermore, a unique triggering mechanism is introduced to coordinate long-term and short-term planning in conjunction with the decision-making module. Finally, a joint impedance controller is applied to control the robot. Notably, the proposed framework is designed to exhibit generalization capabilities. The specific methods for long-term and short-term perception and planning can be flexibly selected and switched based on actual conditions, enabling it to adapt to various robots operating in diverse, dynamic scenarios.

The following sections provide a detailed overview of the proposed HSAC-based long-term and short-term safety control framework and the specific methods employed in this work. Section 3 introduces our perception approach, including long-term and short-term perception. Section 4 presents our planning framework, covering long-term and short-term planning, and discusses the unique Triggering mechanism introduced through convex optimization analysis. In Section 5, the network structure of the proposed Hidden Action Space Heuristic Soft Actor–Critic (HSAC) method is detailed, along with the training and application processes. This section also introduces the implemented curriculum learning mechanism. Finally, Section 6 validates the main contributions of this work and validates the capability and effectiveness of the proposed robot control system in both simulation and real-world environments.

3. Perception method

This section discusses our perception module, which utilizes depth sensors to acquire environmental information for robotic planning. In this work, the depth camera is located beside the robot and is static, forming an eye-out-of-hand configuration. The camera's intrinsic and extrinsic parameters are calibrated using the camera calibration toolbox in MATLAB. The perception component is divided into long-term and short-term perceptions. Long-term perception focuses on acquiring environmental information. In this perception stage, the point cloud of the overall environment is first captured. The point cloud is then preprocessed and clustered using the DBSCAN method [44]. By comparing the Intersection over Union (IoU) of different clusters, the regions to which the point clouds belong are determined, thereby extracting the long-term environmental information.

On the other hand, short-term perception focuses on the immediate obstacle information that has the most significant impact on the robot's safety. One natural and vital information is the distance from the closest obstacle to the robot. To obtain it, pruning is performed based on the minimum enclosing boxes to efficiently compute the nearest point, and finally, Kalman filtering is applied to estimate the position in the real world.

3.1. Long term perception

The workflow of long-term perception is demonstrated in Fig. 2. Using a depth camera, the point cloud of the workspace is obtained. However, the acquired point clouds are dense, noisy, and contain irrelevant information. Therefore, the Region of Interest (ROI) is first identified, and passthrough filtering is utilized to extract it. Subsequently, the point clouds are segmented into appropriately sized voxel blocks, and voxel grid filtering is performed by computing the centroid of each voxel block to achieve downsampling, resulting in a sparse point cloud containing only relevant workspace information.

Further work is needed to cluster the point clouds and identify them distinctly. This work adopts an unsupervised learning method for clustering to fit diverse situations. Based on comparisons across different algorithms, as shown in Fig. 3. Given its promising performance in both effectiveness and real-time capability, the DBSCAN [44] clustering method is applied to group point clouds into distinct clusters, providing structured environmental information. Then, the bounding boxes of the clusters are calculated, and the Intersection over Union (IoU) of the bounding boxes between neighboring frames is computed to track the robot and environment, enabling a structured perception of the global environment. To efficiently parameterize the global environmental information, Octomap is adopted in practice.

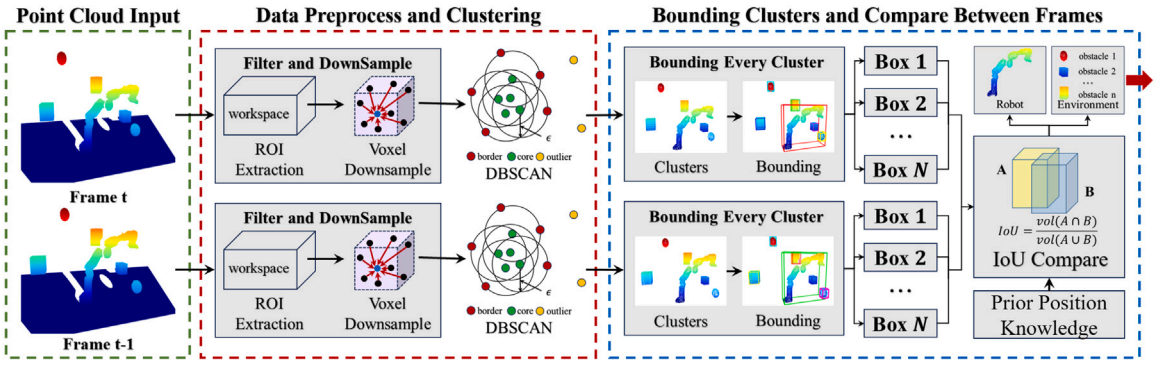


Fig. 2. Workflow of long-term perception. Point cloud derived from the depth camera is extracted to obtain the Region of Interest (ROI). Then, through voxel downsampling, the point cloud is filtered to be sparse. Finally, the DBSCAN clustering algorithm is applied to cluster the point cloud, and the bounding boxes of the clusters are calculated. The Intersection over Union (IoU) of the bounding boxes between neighboring frames is computed and compared to find the best match.

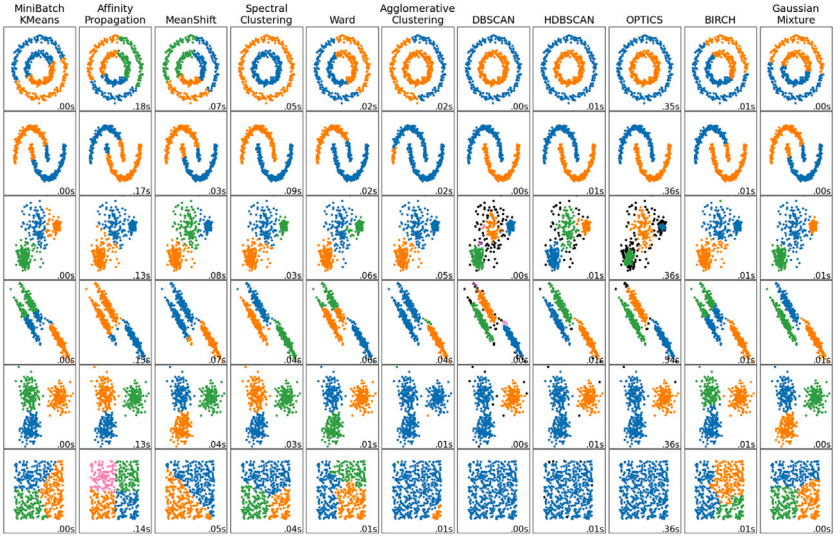


Fig. 3. Comparing different clustering algorithms on toy datasets [45].

Algorithm 1: Long Term Perception

Input: Raw point cloud P_{raw}, robot's initial bounding box b_0
Output: Processed environment point clouds $C_1, C_2, \ldots, C_n$

Check and update b_0 with the current robot configuration
Apply pass-through filtering to extract point clouds of interest P_{filtered} from P_{raw}
Perform voxel down-sampling to obtain downsampled point cloud P_{sparse}
Apply DBSCAN clustering on P_{sparse} to obtain clusters $C_1, C_2, \ldots, C_n$
Set $I_{\max} = 0$
Set $\text{index}_R = 0$
for each bounding cube b_i **do**
    Compute Intersection over Union (IoU) I_i between b_i and tracked bounding box b_0
    if $I_i > I_{\max}$ **then**
        $I_{\max} \leftarrow I_i$
        $b_0 \leftarrow b_i$
        $\text{index}_R \leftarrow i$
    end if
end for
return index_R, $[b_0, b_1 \ldots, b_n]$, $[C_1, C_2, \ldots, C_n]$

3.2. Short term perception

Short-term perception focuses on the information about the nearest distance between the robot and the environment, which is the safety indicator function defined in this work.

First, the formula for calculating the distance from the robot to the closest obstacle is defined. The robot is assumed to be encapsulated by capsules, and the goal is to compute the nearest distance from a given environmental point cloud to these capsules.

Therefore, the robot in short-term perception can be simplified to a set of capsules. For the convenience of calculation, the robot can be further simplified to a set of line segments obtained by connecting adjacent joints of the robot, which is obtained by forward kinematics in this work. This is equivalent to calculating the distance from the point to the line segment and then subtracting the radius of the capsule r_{capsule} as shown in Fig. 4. Let S_{env} denote the set of environmental point clouds, which has been extracted from long-term perception. Thus, the problem reduces to finding the point in the set S_{env} that is closest to a line segment set L_R, which is formed by the robot's components. This relationship can be mathematically expressed as:

$$\mathcal{D}(S_{\text{env}}, L_R) = \min_{p \in S_{\text{env}}} d(p, L_R) - r_{\text{capsule}} \tag{1}$$

where $d(p, L_R)$ represents the distance from point p to the set of line segments L_R. Each line segment in L_R corresponds to a specific link of the robot, making L_R interpretable as the set of robot links.

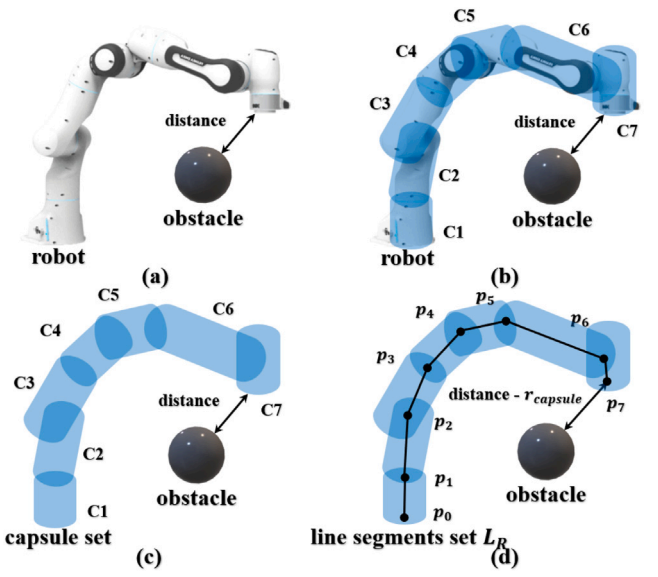


Fig. 4. The robot is encapsulated by capsules. Then, the distance from the robot to the closest obstacle could be simplified to calculate the distance from the point to the line segment set.

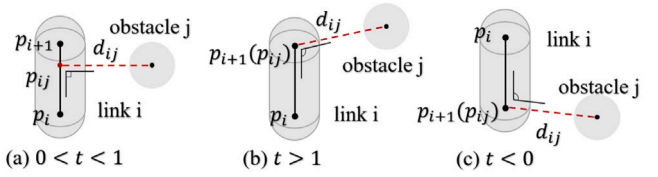


Fig. 5. The figure shows the calculation of the shortest distance from the environmental point cloud to the robot link encapsulated in a capsule.

The term $d(p, L_R)$ can be further defined as:

$$d(p, L_R) = \min_{l_{p_i p_{i+1}} \in L_R} d(p, l_{p_i p_{i+1}}) \tag{2}$$

where $l_{p_i p_{i+1}}$ denotes an individual line segment between two points p_i and p_{i+1} representing a link.

The distance from a point p to a line segment $l_{p_i p_{i+1}}$ can be computed using the following formula:

$$d(p, l_{p_i p_{i+1}}) = \begin{cases} \|p - p_i\| & \text{if } t \le 0, \\ \|p - p_{i+1}\| & \text{if } t \ge 1, \\ \|p - (p_i + t(p_{i+1} - p_i))\| & \text{if } 0 < t < 1, \end{cases} \tag{3}$$

where $t = \frac{(p-p_i)\cdot(p_{i+1}-p_i)}{\|p_{i+1}-p_i\|^2}$ is the projection scalar. This formula accounts for different cases: If $t \le 0$, the closest point is p_i. If $t \ge 1$, the closest point is p_{i+1}. If $0 < t < 1$, the closest point is on the segment, given by $p_i + t(p_{i+1} - p_i)$. The process of this part can be demonstrated in Fig. 5.

This study employs a pruning method based on a minimum bounding box to prevent the loss of real-time performance caused by large-scale point clouds. For each point cloud cluster $[C_1, C_2, \dots, C_n]$, there is a corresponding bounding box $b_1, b_2, \dots, b_n$. Without loss of generality, assume C_1 is the robot's point cloud cluster and b_1 is the robot's bounding box. Let $l_{i,j}^{\min}$ be the minimum distance between any two bounding boxes b_i and b_j, and let $l_{i,j}$ be the minimum distance between the corresponding point clouds C_i and C_j. It follows that:

$$l_{i,j}^{\min} \le l_{i,j} \tag{4}$$

By computing all minimum distances $l_{1,2}^{\min}, l_{1,3}^{\min}, \dots, l_{1,n}^{\min}$ from the clusters to the robot's bounding box and select the smallest minimum distance $l_{1,i}^{\min}$, the short-term perception algorithm can compute $l_{1,i}$ and obtain the threshold $l_{\text{thresh}}^{\min}$. If any minimum distance $l_{1,j}^{\min}$ exceeds this threshold, then the bounding box i should be pruned. From the remaining bounding boxes, select the smallest minimum distance $l_{1,i_1}^{\min}$, compute l_{1,i_1}, and compare it with the current threshold. Update $l_{\text{thresh}}^{\min}$ to the smallest distance and perform pruning. This process continues until no bounding boxes remain. At this point, $l_{\text{thresh}}^{\min}$ represents the minimum distance found. It is also noteworthy that the specific locations of the minimum distances can be recorded, and possible inaccuracies in clustering during long-term planning can be corrected during this process.

Algorithm 2: Short Term Perception

Input: Point cloud clusters C with corresponding bounding boxes $\mathcal{B}$, l_{check}
Output: Minimum distance $l_{\text{thresh}}^{\min}$ and position $x_{\min}$

```
Check and update b_1 with the current robot configuration
Initialize l_thresh^min as ∞
Initialize x_min as none
for each bounding box b_i in the remaining set do
    Compute minimum distance l_{1,i}^min from b_1 to b_i
    if l_{1,i}^min < l_thresh^min then
        Update l_thresh^min to l_{1,i}^min
    end if
end for
while there are remaining bounding boxes do
    Find the index k of the minimum l_{1,i}^min
    Calculate l_{1,k} and x_{1,k} between C_1 and robot
    if l_{1,j} < l_thresh^min then
        if l_{1,j} > l_check then
            Update l_thresh^min to l_{1,k}
            Update x^min to x_{1,k}
        else
            Remove C_k form long term information
        end if
    end if
    Prune bounding box b_k from the remaining set
    for each bounding box b_j in the remaining set do
        if l_{1,j}^min >= l_thresh^min then
            Prune bounding box b_j from the remaining set
        end if
    end for
end while
return l_thresh^min and x^min
```

For bounding boxes, specific directions are selected such that all corresponding edges of the bounding boxes are parallel. Consider a bounding box b_i defined by $\{x_{\min_i}, y_{\min_i}, z_{\min_i}, x_{\max_i}, y_{\max_i}, z_{\max_i}\}$, where $x_{\min_i}$, $y_{\min_i}$, and $z_{\min_i}$ represent the minimum values along the x, y, and z axes, respectively, and $x_{\max_i}$, $y_{\max_i}$, and $z_{\max_i}$ represent the maximum values along these axes. Given two bounding boxes $b_i = \{x_{\min_i}, y_{\min_i}, z_{\min_i}, x_{\max_i}, y_{\max_i}, z_{\max_i}\}$ and $b_j = \{x_{\min_j}, y_{\min_j}, z_{\min_j}, x_{\max_j}, y_{\max_j}, z_{\max_j}\}$, the minimum distance between them can be calculated using the following formula:

$$l_{i,j}^{\min} = \sqrt{d_x^2 + d_y^2 + d_z^2} \tag{5}$$

where d_x, d_y, and d_z are the distances between the bounding boxes along the x-axis, y-axis, and z-axis, respectively. These distances are computed as:

$$\begin{cases} d_x = \max(0, x_i^{\min} - x_j^{\max}, x_j^{\min} - x_i^{\max}) \\ d_y = \max(0, y_i^{\min} - y_j^{\max}, y_j^{\min} - y_i^{\max}) \\ d_z = \max(0, z_i^{\min} - z_j^{\max}, z_j^{\min} - z_i^{\max}) \end{cases} \tag{6}$$

If the bounding boxes intersect (as determined by the Intersection over Union (IoU) during the long-term perception process), the minimum distance $d_{\min}$ is set to zero.

After obtaining the raw data from short-term perception, further filtering of the observations is required to estimate the actual values. This step is necessary in real-world scenarios due to the non-ideal nature of sensors. Specifically, the Kalman filter is employed in this work to re-estimate the measurements. It predicts the system's future state and corrects the prediction using new observations. Through this recursive process, the Kalman filter effectively "filters out" noise and provides a more accurate state estimate. Using this method, short-term obstacle information meeting the specified criteria is obtained. In the subsequent experimental section, the capability and efficiency of this approach will be evaluated.

4. Planning method

This section outlines our planning approach, which focuses on planning within the position loop, with execution carried out using a joint impedance controller at the low level. In our framework, two stages of planning strategies are employed: long-term planning and short-term planning. The long-term planner utilizes parameterized information obtained from long-term perception to generate a collision-free trajectory for the current scene, which is global and static. To endow the robot with dynamic responsiveness, the short-term planner uses parameterized information derived from short-term perception to adjust and refine the reference trajectory. Unlike long-term planning, the short-term plan is local and designed to respond in real-time to dynamic changes in the environment.

A decision-making mechanism based on reinforcement learning is introduced to coordinate these two planners effectively. This decision-making module, which will be elaborated upon in the subsequent Section, is designed to balance the roles of the long-term and short-term planners, ensuring smooth integration and adaptability in dynamic environments.

4.1. Long term planning method

There are numerous mature methods for robot planning in static environments which include sampling-based and optimization-based approaches. In this research, RRT-Connect [20] method is adopted to ensure rapid long-term planning. RRT-Connect is a variant of the Rapidly-exploring Random Tree (RRT) algorithm designed to improve search efficiency, particularly for long-term path-planning tasks. Once the trajectory is planned, cubic spline fitting is applied, followed by uniform frequency sampling. This process ensures a smooth reference trajectory with continuous velocity, which enhances the safety and stability of the robot's movement.

4.2. Short term planning method

Short-term planning serves to make real-time adjustments to the static safe trajectory(reference) generated by long-term planning. This work introduces a safety index function and adopts a Control Barrier Function (CBF) approach to guarantee safety, resulting in an exponentially convergent control strategy that steers the robot back into safe regions. The formulation is simplified to a convex Quadratic Program (QP) whose optimality is analyzed via the Karush–Kuhn–Tucker (KKT) conditions.

Robots operating in dynamic environments must continually adjust their trajectories to avoid unexpected obstacles. Short-term planning refines the reference trajectory from long-term planning by adapting to the current obstacles in real-time. An effective control strategy is required to ensure safety when the robot is in a potentially dangerous state. This work firstly introduces a safety index function ϕ to evaluate the robot's safety.

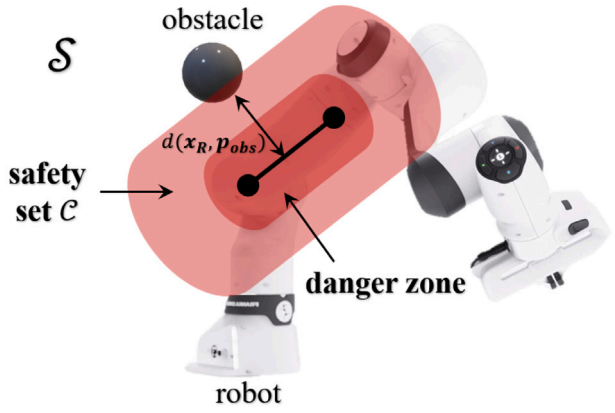


Fig. 6. The set $\mathcal{S}$ is the absolute safety set, where the robot is considered safe. The set $\mathcal{C}$ is the safe set, where the robot is safe but considered in danger of collision.

The safety index function ϕ is defined as:

$$\phi = \epsilon - D(S_{\text{env}}, L_R) = \epsilon - d(\mathbf{x_R}, \mathbf{p}_{\text{obs}}), \tag{7}$$

where ϵ is a predefined safety margin and $d(\mathbf{x_R}, \mathbf{p}_{\text{obs}})$ represents the distance between the robot and the obstacles. Here, $\mathbf{x_R}$ is the vector representation of the robot's current configuration, where the ith element denotes the angle of the ith joint, and $\mathbf{p}_{\text{obs}}$ is the vector representing the current position of the nearest obstacle point. The absolute safety set $\mathcal{S}$ is then defined by $\mathcal{S} = \{\mathbf{x} \in \mathbb{R}^n \mid \phi < 0\}$. Here, the vector $\mathbf{x}$ represents the combined state of the robot and obstacle $\mathbf{x} = [\mathbf{x_R}, \mathbf{p}_{\text{obs}}]$. Within $\mathcal{S}$, the robot is considered safe, and the control strategy follows the reference trajectory from long-term planning.

When $\mathbf{x} \notin \mathcal{S}$ the robot is considered in danger of collision. To address this, this work defines a dangerous margin ϵ_d which is smaller than ϵ and introduce:

$$h(\mathbf{x}) = d(\mathbf{x_R}, \mathbf{p}_{\text{obs}}) - \epsilon_d. \tag{8}$$

Based on the definition of control barrier functions in [30], the safe set is defined as $\mathcal{C} = \{\mathbf{x} \in \mathbb{R}^n \mid h(\mathbf{x}) \geq 0\}$ with the additional requirement that $\frac{\partial h}{\partial \mathbf{x}}(x) \neq 0$, for all x satisfying $h(x) = 0$. Fig. 6 illustrates how these definitions work in our research.

To ensure the safety of the robot, it is necessary and sufficient that the following Control Barrier Function (CBF) condition is satisfied:

$$\dot{h}(\mathbf{x}) \geq -\alpha\big(h(\mathbf{x})\big) \Leftrightarrow \mathcal{C} \text{ is invariant.} \tag{9}$$

Here, α is an (extended) class $\mathcal{K}$ function.

A common choice for the α function is $\alpha(r) = cr$, where c is a positive constant. Moreover, note that in this research $\dot{h}(\mathbf{x}) = \frac{d}{dt} d(\mathbf{x_R}, \mathbf{p}_{\text{obs}}) = -\dot{\phi}$. So the original expression can be reformulated as $-\dot{\phi} \geq -ch(\mathbf{x})$, which implies $\dot{\phi} \leq ch(\mathbf{x})$. Since $h(\mathbf{x}) = d(\mathbf{x_R}, \mathbf{p}_{\text{obs}}) - \epsilon_d = -\phi + (\epsilon - \epsilon_d)$, and because $\epsilon_d < \epsilon$, it follows that $\dot{\phi} \leq -c\phi < -c\phi + c(\epsilon - \epsilon_d) = ch(\mathbf{x})$. This represents a more stringent constraint which guarantees the invariance of the set $\mathcal{C}$:

$$\dot{\phi} \leq -c\phi \Rightarrow \dot{h}(\mathbf{x}) \geq -ch(\mathbf{x}) \Leftrightarrow \mathcal{C} \text{ is invariant.} \tag{10}$$

For the sake of consistency and faster convergence, this work adopts the strict constraint $\dot{\phi} \leq -c\phi$. Simultaneously, this research aims to adhere as closely as possible to the reference trajectory from the long-term planning. To this end, an objective function is defined to minimize the discrepancy between the current state and the reference state. Moreover, the robot's motion must satisfy dynamic constraints, control input, and joint limitations. Accordingly, we formulate an optimization problem that minimizes the difference between the current state and the reference state while simultaneously meeting a variety of

constraints:

$$\begin{aligned} \min_{\mathbf{x_R},\mathbf{u}} \quad & \|\mathbf{x_R}-\mathbf{x_{ref}}\|^2+\delta_c\cdot\|\dot{\mathbf{x}}_\mathbf{R}-\dot{\mathbf{x}}_\mathbf{ref}\|^2 \\ \text{s.t.} \quad & \dot{\phi}\le -c\phi, \\ & \dot{\mathbf{x}}=f(\mathbf{x},\mathbf{u}), \\ & \mathbf{x}\in\mathcal{X}, \quad \mathbf{u}\in\mathcal{U}. \end{aligned} \tag{11}$$

In this formulation, $\mathbf{x_R}$ and $\dot{\mathbf{x}}_\mathbf{R}$ denote the current position and velocity of the robotic system, while $\mathbf{x}_{\text{ref}}$ and $\dot{\mathbf{x}}_{\text{ref}}$ represent the corresponding reference obtained from long-term planning. The term δ_c is a weight factor that balances the importance of minimizing position and velocity errors. The constraint $\dot{\phi}\le -c\phi$ ensures that the robot remains within a safe distance from obstacles. At the same time, $\mathbf{x}\in\mathcal{X}$ and $\mathbf{u}\in\mathcal{U}$ define the feasible space for the robot's motion and control inputs, respectively.

This is a nonconvex optimization problem. In this work, we simplify the problem by delegating the computation of the control input to the downstream joint impedance controller, as shown in Fig. 1, focusing solely on correcting the robot's position loop. Assuming the control frequency f, the duration of each control cycle is given by $\Delta t=\frac{1}{f}$. Assuming $\mathbf{x}'_{\text{ref}}$, $\mathbf{x}'_R$, and $\mathbf{p}'_{\text{obs}}$ are the states from the previous control cycle. Thus, the velocity terms can be approximated as follows:

$$\dot{\mathbf{x}}_{\text{ref}}\approx\frac{\mathbf{x}_{\text{ref}}-\mathbf{x}'_{\text{ref}}}{\Delta t}=(\mathbf{x}_{\text{ref}}-\mathbf{x}'_{\text{ref}})\cdot f, \tag{12}$$

$$\dot{\mathbf{x}}_R\approx\frac{\mathbf{x}_R-\mathbf{x}'_R}{\Delta t}=(\mathbf{x}_R-\mathbf{x}'_R)\cdot f, \tag{13}$$

$$\dot{\mathbf{p}}_{\text{obs}}\approx\frac{\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}}}{\Delta t}=(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}})\cdot f. \tag{14}$$

Therefore, the optimization objective can be rewritten as follows:

$$\min_{\mathbf{x_R}} \quad \|\mathbf{x_R}-\mathbf{x_{ref}}\|^2+\delta_c f^2\cdot\|(\mathbf{x_R}-\mathbf{x}'_R)-(\mathbf{x_{ref}}-\mathbf{x}'_{\text{ref}})\|^2 \tag{15}$$

The CBF constraint can be expressed as:

$$\nabla_{\mathbf{x_R}}\phi(\mathbf{x_R}-\mathbf{x}'_R)+\nabla_{\mathbf{p}_{\text{obs}}}\phi(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}})+\frac{c\phi}{f}\le 0 \tag{16}$$

The low-level controller incorporates a filtering mechanism that guarantees the control inputs remain within their prescribed bounds. Concurrently, this work assumes that the robot does not operate in certain extreme configurations; hence, the constraints on $\mathcal{U}$ and $\mathcal{X}$ can be considered nonrestrictive. This work adopts post-computation checks and scaling of joint angle variations to ensure the robot does not exceed its joint limits. Assuming $\delta=\delta_c f^2$, the original problem is simplified to:

$$\begin{aligned} \min_{\mathbf{x_R}} \quad & \|\mathbf{x_R}-\mathbf{x_{ref}}\|^2+\delta\,\|(\mathbf{x_R}-\mathbf{x}'_R)-(\mathbf{x_{ref}}-\mathbf{x}'_{\text{ref}})\|^2, \\ \text{s.t.} \quad & \nabla_{\mathbf{x_R}}\phi(\mathbf{x_R}-\mathbf{x}'_R)+\nabla_{\mathbf{p}_{\text{obs}}}\phi(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}})+\frac{c\phi}{f}\le 0. \end{aligned} \tag{17}$$

This is a typical Quadratic Programming (QP) problem, which can be solved using the Karush–Kuhn–Tucker (KKT) conditions to obtain the following set of constraint equations:

$$\lambda\ge 0 \tag{18a}$$

$$\lambda\left(\nabla_{\mathbf{x_R}}\phi(\mathbf{x_R}-\mathbf{x}'_R)+\nabla_{\mathbf{p}_{\text{obs}}}\phi(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}})+\frac{c\,\phi}{f}\right)=0 \tag{18b}$$

$$\mathbf{x_R}-\mathbf{x_{ref}}+\delta\left(\mathbf{x_R}+\mathbf{x}'_{\text{ref}}-\mathbf{x}'_R-\mathbf{x_{ref}}\right)+\lambda\left(1+\frac{c}{f}\right)\nabla_{\mathbf{x_R}}\phi=0 \tag{18c}$$

$$\nabla_{\mathbf{x_R}}\phi(\mathbf{x_R}-\mathbf{x}'_R)+\nabla_{\mathbf{p}_{\text{obs}}}\phi(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}})+\frac{c\,\phi}{f}\le 0 \tag{18d}$$

Here, λ represents the Lagrange multiplier.

Case 1: When $\mathbf{x}\notin S$ and $\mathbf{x}\in C$ with $\lambda\neq 0$, the Control Barrier Function (CBF) constraint is active according to the KKT conditions (18b). In this case, the inequality constraint in Eq. (18d) is, in fact, an equality constraint; otherwise, Constraint (18b) would not hold. Thus, the resulting expression for λ can be derived from the KKT conditions as follows:

$$\begin{aligned} \lambda=&\frac{(1+\delta)\left(\nabla_{\mathbf{x_R}}\phi^T\,\mathbf{x}_{\text{ref}}+\nabla_{\mathbf{p}_{\text{obs}}}\phi^T\left(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}}\right)+c\,\phi\right)}{(1+\frac{c}{f})\,\nabla_{\mathbf{x_R}}\phi^T\,\nabla_{\mathbf{x_R}}\phi} \\ &-\frac{\nabla_{\mathbf{x_R}}\phi^T\left(\delta\,\mathbf{x}'_{\text{ref}}+\mathbf{x}'_R\right)}{(1+\frac{c}{f})\,\nabla_{\mathbf{x_R}}\phi^T\,\nabla_{\mathbf{x_R}}\phi} \end{aligned} \tag{19}$$

Then the correction term for $\mathbf{x_R}$ can be expressed as:

$$\begin{aligned} \mathbf{x_R}=&\mathbf{x}_{\text{ref}}+\frac{\delta}{1+\delta}\cdot\left(\mathbf{x}'_R-\mathbf{x}'_{\text{ref}}\right) \\ &-\left(\frac{(1+\delta)\left(\nabla_{\mathbf{x_R}}\phi^T\,\mathbf{x}_{\text{ref}}+\nabla_{\mathbf{p}_{\text{obs}}}\phi^T\left(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}}\right)+c\,\phi\right)}{(1+\delta)\,\nabla_{\mathbf{x_R}}\phi^T\,\nabla_{\mathbf{x_R}}\phi}\right. \\ &\left.-\frac{\nabla_{\mathbf{x_R}}\phi^T\left(\delta\,\mathbf{x}'_{\text{ref}}+\mathbf{x}'_R\right)}{(1+\delta)\,\nabla_{\mathbf{x_R}}\phi^T\,\nabla_{\mathbf{x_R}}\phi}\right) \end{aligned} \tag{20}$$

Case 2: When $\mathbf{x}\notin S$ and $\mathbf{x}\in C$ with $\lambda=0$, the CBF constraint is inactive. In this case, the resulting expression for $\mathbf{x_R}$ can be derived from the KKT conditions (18c) as follows:

$$\mathbf{x_R}=\mathbf{x}_{\text{ref}}+\frac{\delta}{1+\delta}\cdot\left(\mathbf{x}'_R-\mathbf{x}'_{\text{ref}}\right) \tag{21}$$

The derived expression for $\mathbf{x_R}$ are supposed to meet the following condition:

$$\nabla_{\mathbf{x_R}}\phi\,(\mathbf{x_R}-\mathbf{x}'_R)+\nabla_{\mathbf{p}_{\text{obs}}}\phi\,(\mathbf{p}_{\text{obs}}-\mathbf{p}'_{\text{obs}})+\frac{c\,\phi}{f}\le 0, \tag{22}$$

which is the constraint (18d). Otherwise, Eqs. (18c) and (18d) would be incompatible. Therefore, Eq. (18d) can judge whether the CBF constraint is active or inactive. The planning first calculates the result of (21) and then checks Eq. (18d). If Eq. (18d) holds, the CBF constraint is inactive, and the robot can follow the result. Otherwise, if Eq. (18d) does not hold, it indicates that the CBF constraint is active, and the robot needs to adjust its trajectory as shown in Eq. (20).

Case 3: When $\mathbf{x}\in S$, the CBF constraint is removed, and the optimization problem degenerates to:

$$\min_{\mathbf{x_R}} \quad \|\mathbf{x_R}-\mathbf{x_{ref}}\|^2+\delta\,\|(\mathbf{x_R}-\mathbf{x}'_R)-(\mathbf{x_{ref}}-\mathbf{x}'_{\text{ref}})\|^2 \tag{23}$$

The solution is obtained as:

$$\mathbf{x_R}=\mathbf{x}_{\text{ref}}+\frac{\delta}{1+\delta}\cdot\left(\mathbf{x}'_R-\mathbf{x}'_{\text{ref}}\right) \tag{24}$$

After short-term planning, the robot's final trajectory is obtained. In this work, a joint position impedance controller is implemented, interpolating the control signals to 1 kHz based on the sending trajectory, enabling precise control of each joint's motor.

4.3. Trigger mechanism

Since the optimization problem is based on local information and simplified, the solution may not be optimal or safe in all cases. To avoid collisions and local optima, this work introduces a triggering mechanism. This mechanism allows the robot to abandon its current long-term reference trajectory and replan a new long-term reference trajectory while ensuring that the robot refers to a safe trajectory during the replanning process. Since the long-term planner requires non-negligible computation time, the short-term planner must rely on an appropriate temporal reference trajectory during this period to maintain system safety.

In this work, a decision module(this will be introduced in the next Section), which is based on a deep reinforcement learning (DRL) algorithm, is introduced to determine which planner to trigger. When the long-term planner is triggered, the triggering mechanism works for a smooth transition of the long-term reference trajectory. The first step is to obtain a safety reference trajectory during the planning process.

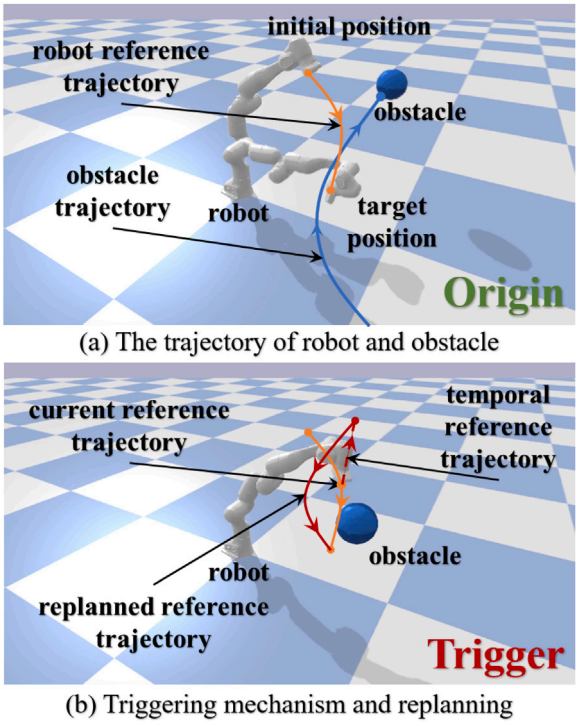


Fig. 7. The triggering mechanism for replanning. Figure (a) shows the robot's origin trajectory and the obstacle's origin trajectory. Figure (b) shows the triggering mechanism and the robot's replanning trajectory. The red dashed line represents the temporal reference trajectory during replanning, while the red solid line represents the reference trajectory from long-term replanning.

Assuming the current state of robot is defined as $\mathbf{x}_{R_0}$. Referring to Cases 1 through 3 in short-term planning, the formulation is partitioned into three components sequentially. It is observed that when the CBF constraint is active, the only distinguishing factor is the third term in (20). Hence, this component represents the direct impact of the CBF safety constraint. Based on the constraint formulation, it can be inferred that its effect is to steer the robot away from obstacles. This process is illustrated in Fig. 7. Consequently, this term serves as a corrective factor facilitating obstacle avoidance. Therefore, this term is utilized to

Algorithm 3: Triggering Mechanism for Long Short Term Planning

Input: Current position $\mathbf{x}_{R_0}$, target position $\mathbf{x}_{\text{end}}$, optimization output $\mathbf{x}_{\text{rect}}$, parameters ρ, N

Output: Long Term Reference Trajectory X_{ref}

Initialize the reference trajectory sequence:
$$X_R = \{\mathbf{x}_{R_i} \mid \mathbf{x}_{R_i} = \mathbf{x}_{R_0} + i \cdot \rho_i \cdot \mathbf{x}_{\text{rect}}, \quad 0 \le i \le N\}$$
Parallel: Start long-term Planning:
$$\text{LongTermPlanning}(\mathbf{x}_{R_N}, \mathbf{x}_{\text{end}})$$
Start short-term Planning:
$$\text{ShortTermPlanning}(X_R)$$
if Both long-term and short-term planning are completed **then**
  Update the reference trajectory:
$$X_{\text{ref}} \leftarrow \text{LongTermPlanning}(\mathbf{x}_{R_N}, \mathbf{x}_{\text{end}})$$
end if
return X_{ref}

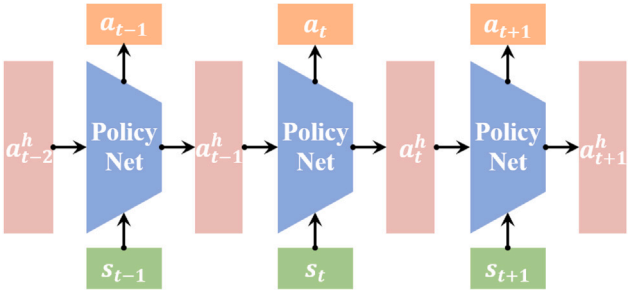


Fig. 8. The workflow of the DRL algorithm with hidden action space.

generate a temporary reference trajectory and defined as $\mathbf{x}_{\text{rect}}$:

$$\mathbf{x}_{\text{rect}} = \left(\frac{(1+\delta)\left(\nabla_{\mathbf{x_R}} \phi^T \mathbf{x}_{\text{ref}} + \nabla_{\mathbf{p}_{\text{obs}}} \phi^T \left(\mathbf{p}_{\text{obs}} - \mathbf{p}'_{\text{obs}}\right) + c\,\phi\right)}{(1+\delta)\,\nabla_{\mathbf{x_R}} \phi^T \nabla_{\mathbf{x_R}} \phi} - \frac{\nabla_{\mathbf{x_R}} \phi^T \left(\delta\,\mathbf{x}'_{\text{ref}} + \mathbf{x}'_R\right)}{(1+\delta)\,\nabla_{\mathbf{x_R}} \phi^T \nabla_{\mathbf{x_R}} \phi} \right) \tag{25}$$

The safety reference trajectory is then defined as:

$$\mathbf{x}_{R_i} = \mathbf{x}_{R_0} + i \cdot \rho_i \cdot \mathbf{x}_{\text{rect}}, \quad 0 \le i \le N \tag{26}$$

where $\mathbf{x}_{\text{rect}}$ plays a crucial role in pulling the manipulator back to a safe region. Here, N represents the number of reference trajectory points to be generated, and ρ_i is a scaling factor to scaling the magnitude of joint movements to avoid joint over-limit conditions.

In this setup, $\mathbf{x}_{R_N}$ is the starting point for the new long-term reference trajectory. During the long-term planning process, the short-term planner avoids obstacles until the target is reached.

5. Decision method

This chapter introduces a DRL-based scheduling mechanism to intelligently select evasive strategies under dynamic conditions. Building on the SAC algorithm [46,47], this study proposes the Hidden Action Heuristic Soft Actor–Critic (HSAC), which employs hidden action spaces to better capture non-Markovian characteristics. This approach allows efficient coordination between short-term and long-term planning, minimizing unnecessary replanning while proactively adjusting to complex scenarios. To handle the strict no-collision requirement without training difficulties, this study adopts a curriculum learning strategy [48], gradually guiding the network to adapt and learn the original task effectively.

5.1. Soft actor–critic with hidden action heuristic (HSAC)

In this work, the designed algorithm aims to act as a decision-maker that regulates the relationship between long-term and short-term planning. To achieve this, specific features are extracted as system inputs s_t, including the current distance, the derivative of distance over time, the action-related information, and their previous time-step value. The system outputs the action a_t. Specifically, the action a_t in this work is defined as the probabilities for executing long-term planning and short-term planning within the interval [0,1], and the action with the higher probability is selected for execution.

To achieve more intelligent decision-making, this work adopts a deep reinforcement learning (DRL) algorithm to train the decision-making model. However, the standard DRL algorithm cannot model the problem's non-Markovian properties directly, which may lead to loss of performance. To address these limitations, inspired by the classical RNN architecture, this work attempts to model the non-Markovian properties using a hidden state. Specifically, the hidden state is a latent variable that, while not directly observable, can be inferred from the

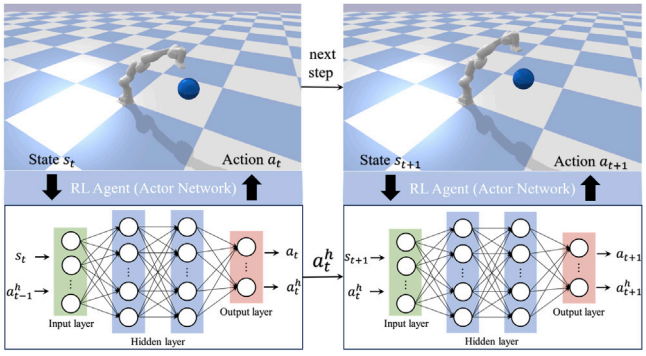


Fig. 9. The workflow of the DRL algorithm with hidden action space in this research.

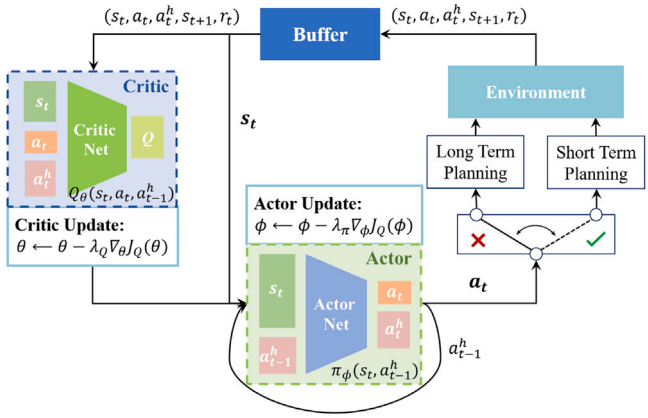


Fig. 10. Workflow of HSAC: this process outlines the specific steps of HSAC, which were employed in our work. More techniques are applied in the complete algorithm.

available data. It is utilized as part of both the input and the output. The policy network takes the current state and the hidden action as input and produces the next action to be taken along with the hidden action for the next time step, as shown in Fig. 8. This process is similar to a recurrent neural network (RNN) architecture, in which the hidden state is updated at each time step based on the current input and the preceding hidden state. Therefore, it is capable of capturing temporal information. However, it is not the same as RNN. The architecture in this work does not first regress the hidden state h and then use it to calculate the regression result or pass to the next time step. Instead, it directly regresses the hidden action pass and the current action to take. Compared to LSTM, this model structure is more linear and involves fewer parameters, demonstrating greater robustness when deployed in real-world environments in our research. Subsequent experiments have also shown significant performance improvements, making it highly suitable for the current application scenario.

Due to its strong policy stability, high adaptability, and robustness[49], Soft Actor–Critic (SAC) often outperforms other algorithms in robotic operations requiring deployment in real-world environments[51]. In this work, SAC is adopted as the foundation for training the decision-making model. And improvements are made to the original SAC algorithm to enhance its performance as discussed below. Therefore, this work introduces the hidden action Heuristic soft actor–critic (HSAC) approach. In this approach, the actor's output now consists of both a_t and a_t^h. Here, a_t directly participates in action decision-making, determining whether the system engages in long-term or short-term planning. In contrast, a_t^h, referred to as the hidden action, does not directly influence the decision process but represents an internal decision of the actor. It acts as a latent signal, akin to an internal acknowledgment such as, "This decision seems risky; I should be more cautious in the next control cycle". These "hidden actions" play a crucial role in the next cycle's decision-making process, functioning as part of the actor's input .The overall workflow of HSAC in this research is illustrated in Fig. 9.

In the training process, the actor network is same as the network illustrated in Fig. 9. The critic is also augmented to access the hidden action a_t^h as its input, enabling it to estimate the value of current state–action pairs better and guide the actor toward more optimal behaviors. In other words, the critic network's target is to evaluate the action value of $Q(s_t, a_t, a_t^h)$, which is the Q-value of the current state–action pair within hidden actions.

The specific process of the interaction between the RL agent and the environment is as follows. The robot is given a moving task in an environment with dynamic obstacles. Obtain the current state s_t from the simulation environment and feed it into the decision module. The decision module outputs the current action a_t and the hidden action a_t^h. Then, a_t is executed in the environment, which returns the subsequent state s_{t+1} and reward r_t. Next, the tuple $(s_t, a_t, a_t^h, s_{t+1}, r_t)$ is stored in the experience replay buffer. Finally, samples from the replay buffer are used to update the parameters of both the policy network and the value network. Both networks employ a multilayer perceptron (MLP) architecture. This process is iteratively executed, with additional samples being drawn and processed until the network converges, as illustrated in Fig. 10. The following parts detail the design of the loss functions and the parameter update procedures. The loss functions used in this method are defined as follows:

Critic Loss Function: The Critic loss function is updated based on the Bellman equation and includes the hidden action a_t^h as an input, which helps evaluate the potential information behind the current decision.

$$J_Q(\theta_i) = \mathbb{E}_{(s_t, a_t, a_t^h, r_t, s_{t+1}) \sim D} \left[\left(Q_{\theta_i}(s_t, a_t, a_t^h) - y_t \right)^2 \right] \tag{27}$$

where:

$$y_t = r_t + \gamma \mathbb{E}_{a_{t+1}, a_{t+1}^h \sim \pi(\cdot|s_{t+1})} \left[\min \left(Q_{\theta_1'}(s_{t+1}, a_{t+1}, a_{t+1}^h), Q_{\theta_2'}(s_{t+1}, a_{t+1}, a_{t+1}^h) \right) \right] \tag{28}$$

This equation demonstrates the contribution of the hidden action a_t^h, where it is incorporated into the Q-function evaluation and thereby influences the update. The update process:

$$\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} J_Q(\theta_i) \quad i = 1, 2 \tag{29}$$

Then, a soft update strategy is applied to update the target network:

$$\theta_i' \leftarrow \tau \theta_i + (1 - \tau) \theta_i' \quad i = 1, 2 \tag{30}$$

Policy Loss Function: Similarly, the policy network is also influenced by the hidden action a_t^h, and the policy optimization must account for a_t^h in addition to the standard action a_t.

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D} \left[\mathbb{E}_{a_t, a_t^h \sim \pi_\phi} \left[\log \pi_\phi(a_t, a_t^h | s_t, a_{t-1}^h) - Q_{\theta_1}(s_t, a_t, a_t^h) \right] \right] \tag{31}$$

Then, the policy network is updated:

$$\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi) \tag{32}$$

When updating the policy, this loss function considers not only the standard action a_t but also evaluates the implicit information behind the action selection through a_t^h. This additional component ensures the policy accounts for the latent decision factors encoded in the hidden action space. The proposed algorithm is outlined as algorithm 4.

The HSAC algorithm can capture the task's non-Markovian propriety and long-term dependencies. Furthermore, it enhances decision-making continuity and consistency.

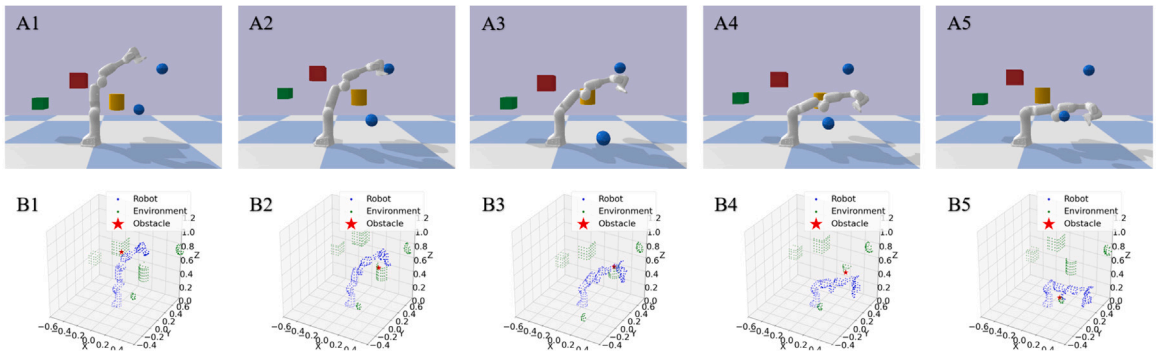


Fig. 11. A comprehensive perception method testing result: the top images show RGB photographs of the experimental environment, while the bottom images visualize the perception results with the environment in green, the robot in blue, and the nearest obstacle in red.

Algorithm 4: Soft Actor–Critic with Hidden Action-space Heuristic (HSAC)

Input: Initial parameters θ, θ', ϕ
Output: Optimized parameters θ_1, θ_2, ϕ
Initialize target network weights: $\theta'_1 \leftarrow \theta_1$, $\theta'_2 \leftarrow \theta_2$
Initialize an empty replay buffer $D \leftarrow \emptyset$
for each iteration **do**
  for each environment step **do**
    Sample action $a_t, a_t^h \sim \pi_\phi(a_t, a_t^h | s_t)$ from the policy network
    Sample next state $s_{t+1} \sim P(s_{t+1}|s_t, a_t)$ from the environment
    Store transition $(s_t, a_t, a_t^h, r_t, s_{t+1})$ in replay buffer D
  end for
  for each gradient step **do**
    Update the Q-function parameters for $i = 1, 2$:
$$\theta_i \leftarrow \theta_i - \lambda_Q \nabla_{\theta_i} J_Q(\theta_i)$$
    Update the policy weights:
$$\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi)$$
    Update the target network weights for $i = 1, 2$:
$$\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i$$
  end for
end for

5.2. Curriculum learning strategy

To train the proposed HSAC method, this study focuses on both task completion and execution efficiency, employing a curriculum learning strategy to increase collision penalties and ensure safety gradually. Specifically, a combination of sparse and dense rewards is used. The reward function is designed to encourage the robot to complete tasks efficiently while avoiding collisions. The reward function consists of a sparse reward and a dense reward. The sparse reward is given when the robot completes the task or collides with an obstacle, while the dense reward is provided during task execution to guide the robot's behavior. The overall reward is defined as:

$$r = r_s + r_d \tag{33}$$

Given a task that requires the robot to move from its current initial position to a designated target position. Upon task completion, a one-time reward is given, defined as $r_s = 4000 - 10 \cdot N_{\text{step}}$, where the reward is negatively correlated with the total number of steps—encouraging faster completion.

Dense rewards are also provided during task execution, which often helps accelerate network training. Specifically, short-term planning incurs a penalty of −1 during each control cycle. If the robot's distance to an obstacle becomes too close and enters a safety–critical zone, an additional penalty of −3 is applied. Furthermore, if long-term planning is triggered, the robot incurs an additional penalty of −200, encouraging the robot to avoid frequently switching trajectories and maximize efficiency.

$$r_d = \begin{cases} -1, & \text{if short-term planning and } d > \epsilon, \\ -3, & \text{if short-term planning and } d \le \epsilon, \\ -200, & \text{if long-term planning is triggered} \end{cases} \tag{34}$$

where d is the distance between the robot and the obstacle, and ϵ is a threshold distance.

For collision handling, a curriculum learning strategy is adopted over multiple stages. The collision penalty is initially set to a relatively low value, such as $r_c = -10^3$, to allow the network to develop a feasible policy. As training progresses, the penalty increases by an order of magnitude in each stage, reaching $r_c = -10^4$ and later $r_c = -10^5$ in the final stage.Therefore, the sparse reward in different stage N_{step} and step $N_{\text{stage}} \in \{1, 2, 3\}$ is defined as:

$$r_s = \begin{cases} 4000 - 10 \cdot N_{\text{step}}, & \text{if task completed,} \\ -10^{2+N_{\text{stage}}}, & \text{if collision occurred.} \end{cases} \tag{35}$$

Additional stages with stricter penalties can be introduced to meet higher safety standards if required.

6. Experimental results

The experiments are divided into four parts. First, the perception system is evaluated separately, with its performance compared against conventional methods. Next, the training results of the proposed Hidden Soft Actor–Critic (HSAC) approach are presented alongside other reinforcement learning techniques (SAC, PPO, TD3), highlighting the benefits of introducing the hidden action space and employing curriculum learning. Subsequently, the proposed system is validated in simulation by reproducing and comparing it against the aforementioned methods across 10,000 randomly generated safety scenarios, demonstrating superior performance in terms of both safety and efficiency. Finally, the entire system is tested under real-world conditions, conducting three distinct experiments to confirm its practicality and robustness.

Our perception system uses Open3D [49], while the DRL agent leverages Stable Baselines3 [50], with simulations carried out in PyBullet and path planning via OMPL [51]. For real-world tests, this study integrates a Franka Emika Panda robotic arm, Kinect DK sensor, ROS, and MoveIt, demonstrating the system's effectiveness in practical applications.

6.1. Perception experiments

Our proposed method through simulation experiments are evaluated using a system equipped with a 12th Gen Intel® Core™ i5-12500H

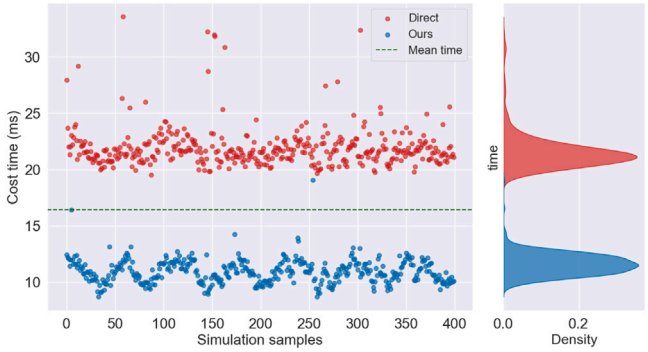


Fig. 12. The comparison between our method and direct calculating method.

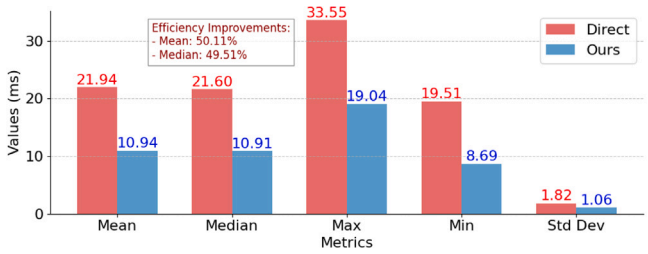


Fig. 13. The statistical comparison between our method and direct calculating method.

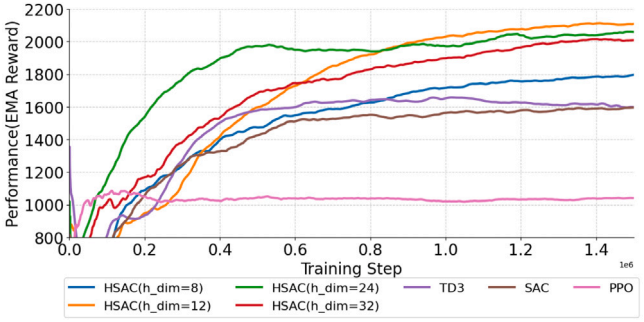


Fig. 14. The EMA reward between different methods during the training process.

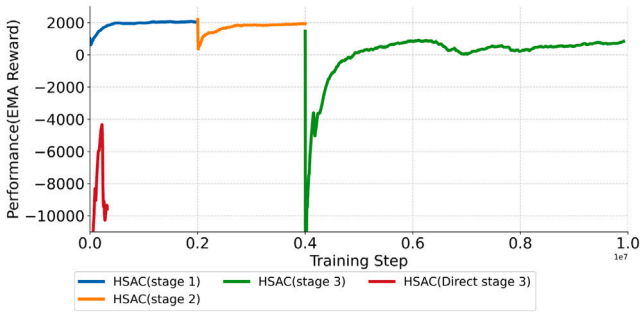


Fig. 15. Curriculum learning h_dim = 24 achieves successful training by guiding the model through structured stages, while direct training often stagnates after a certain period. Logs reveal that direct training causes the actor to over-prioritize long-term planning, leading to continuous replanning without forward progress, ultimately hindering training advancement.

processor (64-bit operating system) with 16 GB of RAM. No GPU acceleration was utilized. The parameters were set as follows: the Region of Interest (ROI) was defined with $x_{\min} = -1$, $x_{\max} = 1$, $y_{\min} = -0.7$, $y_{\max} = 0.7$, $z_{\min} = 0.05$, and $z_{\max} = 1.5$. For passthrough filtering, a voxel size of 0.04 was used for downsampling, and the DBSCAN clustering parameters were set to $\epsilon = 0.08$ and min_samples = 8.

Fig. 11 shows an overview of the perception process of the virtual experiment.

The time efficiency of our method was compared with traditional approaches, as depicted in Fig. 12. In the direct method, after applying the same passthrough filtering and downsampling process, the classification of points and nearest obstacle detection by directly calculating the distances between points and each robot link.

The experimental results demonstrate that the proposed method significantly outperforms the Direct method regarding computational efficiency and consistency, as shown in Fig. 13. The mean computation time of the proposed method is 10.94 ms, compared to 21.94 ms for the Direct method, achieving an efficiency improvement of approximately 50.11%. Similarly, the median time is reduced from 21.60 ms to 10.91 ms, with an improvement of 49.51%. The proposed method exhibits minor variability, with a standard deviation of 1.06 ms compared to 1.82 ms for the Direct method, indicating more consistent performance. Additionally, the proposed method's maximum and minimum computation times (19.51 ms and 8.69 ms) are significantly lower than those of the Direct method (33.55 ms and 19.04 ms). These statistical findings align with the trends observed in Fig. 12.

6.2. Deep reinforcement learning for decision mechanism

The entire experimental system was first constructed in the simulation environment. Franka Emika Panda was selected as the robot platform, and the simulation environment was built using PyBullet. In this environment, the robot is given a task to move from its initial position to a target position while avoiding obstacles. The obstacle is a moving ball that moves along a random trajectory. Whenever the environment is reset, a novel obstacle trajectory is generated by randomly sampling points in space and fitting a smooth trajectory using cubic spline interpolation. This approach ensures that the obstacle's trajectory is distinct in every experiment, with position, velocity, and acceleration remaining continuous. The simulation environment is shown in Fig. 7. The target of DRL training is to learn a policy to determine whether to trigger long-term or short-term planning based on the current state of the robot and the environment to enable the robot to reach the target position effectively while ensuring safety.

Within the same environment, the proposed HSAC method (with varying hidden dimensions) was compared against standard reinforcement learning algorithms, including SAC, PPO, and TD3. These algorithms are used or complemented based on OpenAI's Stable Baselines3 library. The training is executed on an NVIDIA GeForce RTX 3060 Laptop GPU. The RL agent generates a decision at every step to determine whether to execute long-term planning or short-term planning. An episode terminates upon reaching the target position or when a collision occurs, after which the environment is reset. A total of 750 training rounds were conducted, each consisting of 2000 steps, and the training time for a single round is approximately 25 h. The training is divided into three stages for subsequent curriculum learning, as described in the reinforcement learning section. In the first stage, the collision penalty is set to -10^3, in the second stage to -10^4, and in the third stage to -10^5. The number of training steps is adjusted to 2×10^6, 2×10^6, and 6×10^6, respectively, resulting in a total training time of approximately 137.5 h.

The final training results are presented in Fig. 14. The results show that the HSAC method outperforms the other algorithms. Further analysis reveals that performance initially improves with increasing hidden dimensions but stabilizes and slightly declines beyond a certain point. This indicates that the HSAC method effectively models and learns non-Markovian dynamics, requiring a specific range of sizes to encode these properties. However, huge hidden dimensions can lead to training and convergence difficulties, highlighting the importance of balancing model complexity.

Table 1
Performance metrics comparison across methods: To comprehensively evaluate the effectiveness of our proposed method, this study considered several performance metrics: Success rate, representing the proportion of collision-free trajectories as a measure of task safety; Win rate, indicating our method's superiority in competitive scenarios where some methods failed, and some methods succeed; Average time, reflecting the meantime of all trajectories; and Consistent success average time, providing a fair temporal comparison for cases where all methods succeeded.

Methods	Performance		Efficiency	
	Success rate (%)	Win rate (%)	Avg time (Steps)	Consistent avg time (Steps)
Ours	**98.32**	**96.89**	119.54	**106.73**
SAC aware	86.52	73.65	108.22	106.97
Time scaling	66.97	35.15	117.35	153.54
Trigger	84.10	68.89	**106.91**	119.07
Token bucket	87.22	75.03	131.77	144.02

Moreover, by employing a curriculum learning strategy, incremental convergence across three training stages was observed, ultimately resulting in the desired network. Compared to direct learning method in Fig. 15 which fails to converge to a feasible policy, the results emphasize the importance of curriculum learning in improving convergence and overall performance.

6.3. Simulation experiment comparison

To evaluate the proposed method, it was compared against three traditional approaches – Time Scaling, Trigger, and Token Bucket – as well as a safety-aware approach based on SAC. A random sample of 10,000 obstacle interference trajectories was generated, and the robotic arm was tested under consistent scenarios across all methods to ensure fair comparisons.

In implementing these methods, the reference trajectory was generated using OMPL's RRT-Connect algorithm with identical parameter settings across all trials. Control Barrier Function (CBF) parameters were kept consistent as well. A threshold of 0.4 m was defined as the danger zone indicator, while a safety margin of 0.2 m was adopted. For the Time Scaling method, a conservative strategy was employed: when obstacles were within 0.4 to 0.8 m of the robotic arm, the trajectory was adjusted to slow down, and the arm completely stopped if the distance was less than or equal to 0.4 m. In the selection of parameters for the Control Barrier Function (CBF), the frequency is set to $f = 50$ Hz, $\delta = 0.5$, $c = 0.4$, and $\epsilon = 0.4$ m, which represents our defined safe margin. Table 1 summarizes the performance results.

The results demonstrate that our proposed method achieves significantly higher success in the tested 10,000 trajectories. Additionally, our method consistently outperforms others in competitive scenarios, achieving superior win rates. Notably, even in cases where all methods succeeded, our approach delivered the best temporal efficiency. This highlights the ability of our method to ensure safety while maintaining efficiency, substantiating its intelligent design.

Three examples are selected in Fig. 16 to further analyze this phenomenon. Across the three cases, our method demonstrates robust adaptability and intelligent coordination. In Case 1, it effectively handles random obstacle collisions with cautious and effective behavior, outperforming SAC and manual triggers. In Case 2, it balances caution and efficiency through localized adjustments guided by HSAC. In Case 3, it achieves optimal timing in aggressive scenarios, ensuring safety while maintaining efficiency. These results validate the versatility of our approach in dynamic conditions.

6.4. Experiment in the real world

The safety control system was implemented in a real-world environment. On the hardware side, a 7-DOF Franka Emika Panda robot was used for manipulation tasks, and a Kinect DK served as the perception module. Depth images were captured using the NFOV_UNBINNED mode of the Kinect DK. The overall hardware architecture is illustrated in Fig. 17.

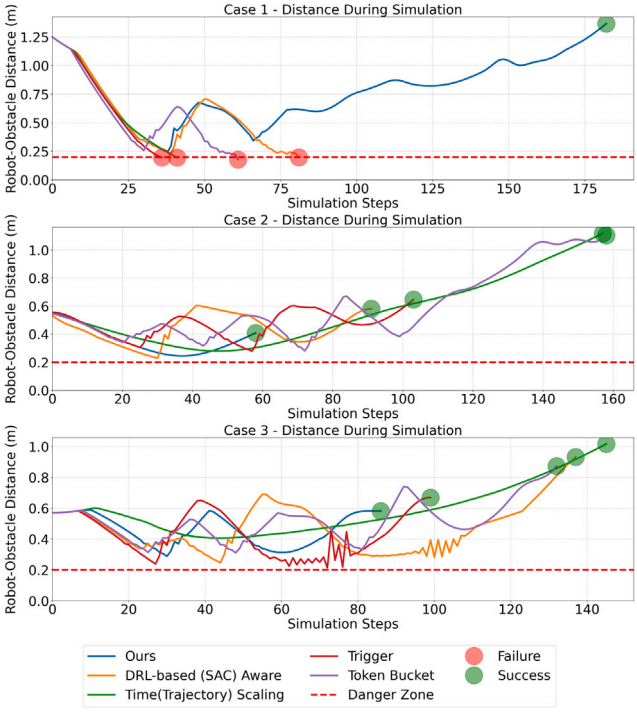


Fig. 16. In case 1, under scenarios with random obstacle collisions, manually designed triggering methods fail to respond intelligently, particularly to fast-approaching obstacles. While the SAC method shows some adaptability, it misjudges subsequent situations. Our method, leveraging the hidden action space to encode past information, exhibits cautious and effective behavior. In case 2, unlike purely cautious approaches, our method uses historical encoding via the hidden action space to guide decisions. While other methods adopt aggressive trajectory adjustments, our approach performs localized adjustments, ensuring safety with optimal efficiency. In case 3, all methods employ aggressive adjustments, but our approach demonstrates better timing, balancing safety and efficiency.

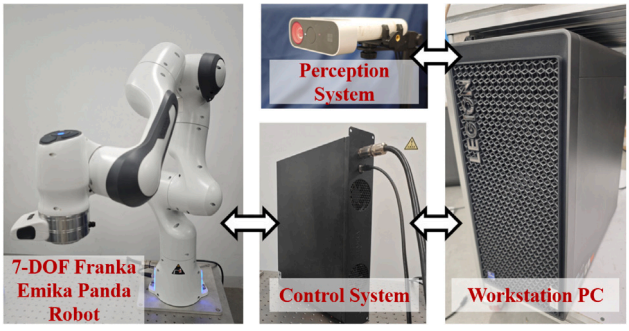


Fig. 17. The hardware configuration in the real world experiment.

On the software side, ROS and MoveIt were employed to construct the system and facilitate information exchange between modules. The long short term perception algorithm runs at a frequency of 30 Hz with parameters set as voxel_size = 0.01, eps = 0.04, min_samples = 64, $x_{\min} = -0.6$, $x_{\max} = 1$, $y_{\min} = -0.6$, $y_{\max} = 1$, $z_{\min} = 0.1$, $z_{\max} = 1.5$. In the selection of parameters for the Control Barrier Function (CBF), the frequency is set to $f = 50\,\text{Hz}$, $\delta = 0.5$, $c = 0.4$, and $\epsilon = 0.4\,\text{m}$. The Kalman filter is configured with a time step of $\Delta t = 1/30\,\text{s}$, corresponding to the sensor sampling frequency. The state transition matrix F describes the linear relationship between position and velocity, while the observation matrix H indicates that the sensor only measures position. The observation noise covariance matrix R is set to $\text{diag}(0.001, 0.001, 0.001)$ based on sensor noise characteristics, and the process noise covariance matrix Q is derived assuming an acceleration noise of $0.5\,\text{m/s}^2$. The initial error covariance matrix $P_{k|k}$ is configured with variances of 1.0 for both position and velocity.

The decision-making module utilizes the three-stage trained HSAC algorithm, operating at 50 Hz. Long-term planning leverages the RRT-Connect algorithm, using the latest long-term perception data as a reference and operating at 1–2 Hz. Short-term planning runs at a fixed frequency of 50 Hz. At the low level, an impedance controller ensures 1 kHz control for precise execution. The overall framework is illustrated in Fig. 1.

Experiments Setup: Three experiments were designed to validate the proposed framework's performance in real-world physical environments. The first experiment investigated the system's response to static obstacles. Specifically, a static obstacle, absent during the robot's initial task planning, is suddenly introduced into its motion trajectory to test its reaction capabilities. This configuration was chosen because if the obstacle were present before, the robot would naturally avoid it through long-term planning, which would not effectively validate the framework's performance.

The second experiment features dynamic obstacles introduced to evaluate the robot's ability to react in dynamic environments. After the robot begins its motion, an experimenter moves a blue rod rapidly toward the robot's current position. This action, called a 'hit,' entails the rod traveling from outside the robot's workspace (as indicated by the green region in the following figure) to the robot's present position within two seconds. This experiment evaluates whether the robot can promptly evade obstacles in a dynamic environment.

The third experiment combines static and dynamic obstacles, introducing both types simultaneously to test the robot's robustness and adaptability under complex interference scenarios. Due to the influence of dynamic obstacles, the static obstacle can be positioned before the robot's motion, which is different from the first experiment.

Experiments Results: The results of the three experiments validate the proposed framework's capability for safe control in dynamic environments and demonstrate its practical applicability in real-world scenarios. The time-series snapshots and distance over time plots for each experiment are shown in Figs. 18–23. This research provides time-series snapshots and plots of distance variation over time for each experiment. In the time-series snapshots, the left panel shows the robot at different times, while the upper-right panel displays the corresponding Rviz view. It can be observed that when long-term planning is triggered, the Octomap is updated. In the distance versus time plots, the green region indicates that the obstacle is safely located outside the robot's workspace (set to 1 m for simplicity), and the red line delineates the boundary of the danger zone; if the distance falls below this threshold, a collision is deemed to have occurred.

In experiment 1, the robot successfully maintains the distance above the danger line. Due to the static nature of the obstacle, our method adopts a more "aggressive" strategy, making slight trajectory adjustments and triggering replanning only when the distance to the danger zone is within a few centimeters.

In experiment 2, during three sudden hits, our method successfully avoids collisions and triggers replanning. Compared to Case 1, the

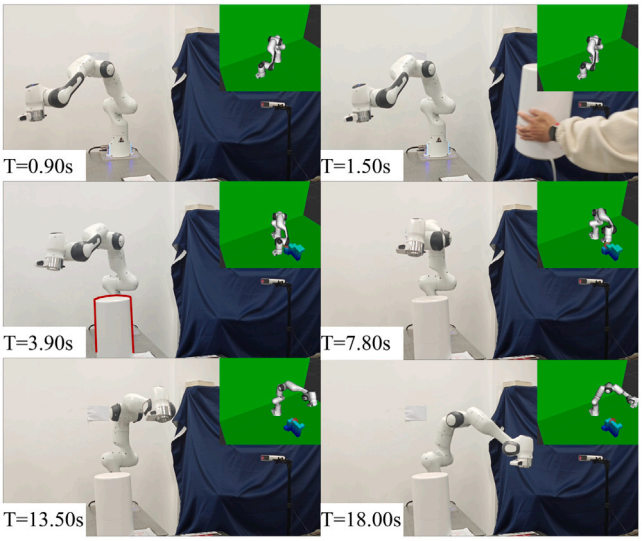


Fig. 18. Time-series snapshots of the real-world experimental scenario for Case 1.

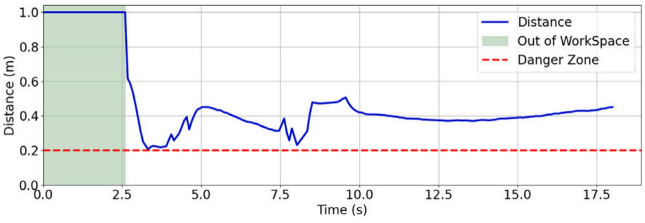


Fig. 19. Distance between the robot and the obstacle over time in Case 1.

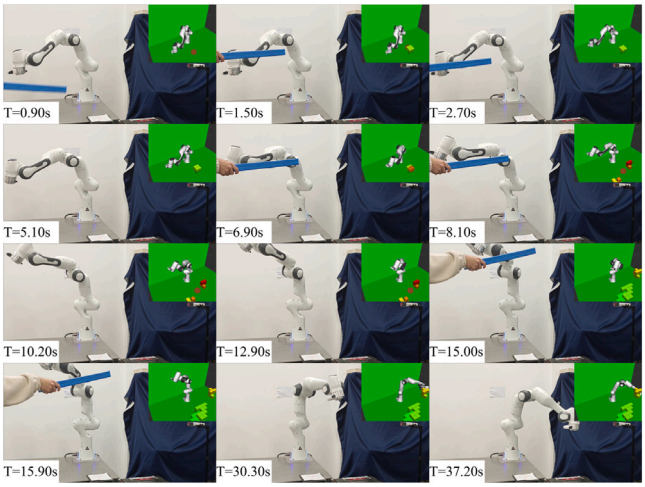


Fig. 20. Time-series snapshots of the real-world experimental scenario for Case 2. The robot faces three sudden hits during this experiment.

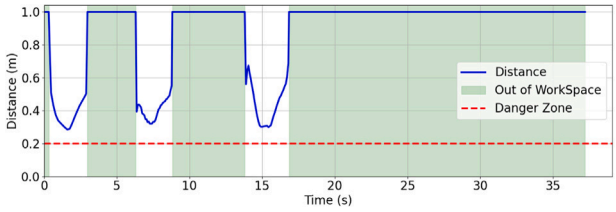


Fig. 21. Distance between the robot and the obstacle over time in Case 2.

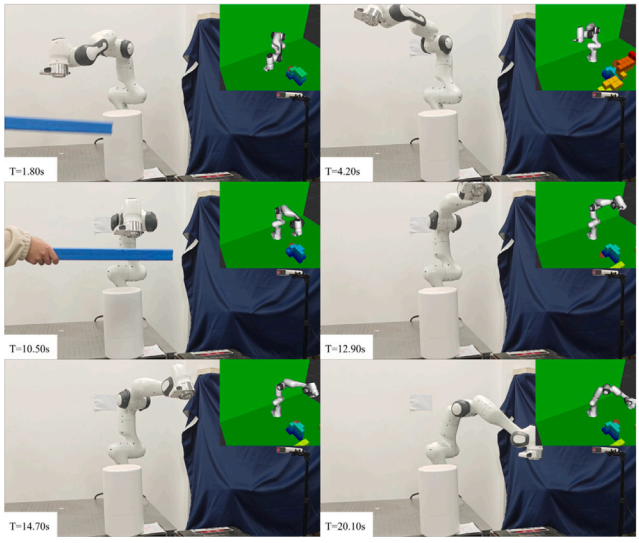


Fig. 22. Time-series snapshots of the real-world experimental scenario for Case 3, which involves both static and dynamic obstacles.

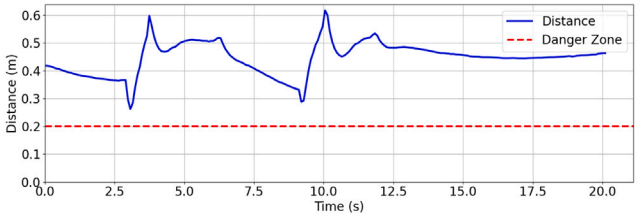


Fig. 23. Distance between the robot and the obstacle over time in Case 3.

method adopts a more cautious strategy. Replanning is observed at approximately 30 cm from the obstacle, ensuring safety while maintaining efficiency.

In experiment 3, with both static and dynamic obstacles present, the robot successfully avoids collisions through cautious trajectory adjustments. This experiment highlights the intelligent decision-making capability of our method, balancing safety and efficiency across diverse scenarios.

The results of the three experiments validate the proposed framework's capability for safe control in dynamic environments and demonstrate its practical applicability in real-world scenarios.

7. Discussion

In this study, we introduced an integrated robotic safety control framework that spans perception, decision-making, planning, and execution.

By combining long-term and short-term perception strategies, approximately a 50% reduction in computational overhead was achieved, along with improved robustness against noise, facilitating a smooth sim-to-real transition. Future efforts could focus on exploring more advanced clustering or adaptive filtering techniques to further enhance reliability under dynamic conditions.

Our reinforcement learning experiments showed that increasing the hidden dimension enriched the encoding of non-Markovian dynamics, improving and stabilizing performance. However, overly large hidden dimensions produced diminishing returns, highlighting a complexity-performance trade-off. Future work may focus on automated dimensionality selection, hybrid architectures, or domain-informed regularization to optimize representational capacity.

Comparative analysis against four baseline methods across 10,000 randomized simulation trials underscores our framework's superiority in maintaining safety while achieving task objectives efficiently. The intelligent behaviors highlighted in three representative cases reflect the method's ability to adaptively manage competing demands for safety and efficiency.

Finally, our successful transfer of the proposed framework to a real-world setting not only substantiates the simulation-to-reality pipeline but also speaks to the framework's capacity for safe control under non-ideal, dynamic circumstances. Although the physical experiments demonstrated strong performance, practical deployments often demand constant monitoring, maintenance, and updates to handle real-world uncertainties and sensor degradation over time. Future research should consider the integration of online learning strategies or continual adaptation mechanisms to refine performance as environmental conditions evolve.

8. Conclusion

This work presents a robotic safety control system that integrates perception, decision-making, and planning into a cohesive framework. In the perception module, a long-term and short-term perception strategy is proposed, where long-term perception decomposes point clouds into structured information, and short-term perception extracts the nearest obstacle information from the environment. Experimental results demonstrate significant computational efficiency improvements while enhanced noise robustness, making the perception system highly suitable for real-world applications. In the decision-making module, a reinforcement learning-based approach is employed to train the robot's decision policy, addressing the non-Markovian characteristics of the problem by introducing a hidden action space to encode non-Markovian dynamics. This method achieves substantial performance improvements without modifying the original network structure. The planning module employs a long short-term framework, where long-term planning uses sampling-based methods to provide reference trajectories for static environments, and short-term planning optimizes these trajectories using dynamic environmental information. A unique triggering mechanism is introduced to effectively coordinate long-term and short-term planning interaction. Comprehensive comparisons with other methods, conducted across 10,000 randomized obstacle safety control experiments, validate the proposed system's superior safety control capabilities and efficiency. Furthermore, the successful deployment of this system in real-world physical environments, including tests under various dynamic scenarios, demonstrates its practical feasibility and application potential.

CRediT authorship contribution statement

Yixuan Liang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Investigation, Formal analysis, Conceptualization, Project administration, Methodology, Data curation. **Ze Wang:** Funding acquisition, Conceptualization, Writing – review & editing, Supervision, Resources, Project administration, Methodology. **Yunan Wang:** Formal analysis, Conceptualization, Writing – review & editing, Visualization, Methodology, Funding acquisition. **Jichuan Yu:** Writing – review & editing, Software, Methodology, Formal analysis, Conceptualization. **Jizhou Yan:** Software, Investigation, Data curation, Validation. **Shize Lin:** Software, Data curation, Visualization, Validation. **Zhao Jin:** Investigation, Validation, Data curation. **Jiuru Lu:** Visualization, Supervision, Data curation. **Chuxiong Hu:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB4302003, and in part by the National Natural Science Foundation of China under Grants 52205532 and 624B2077.

Data availability

Data will be made available on request.

References

[1] Guang-Zhong Yang, Jim Bellingham, Pierre Dupont, Peer Fischer, Luciano Floridi, Robert Full, Neil Jacobstein, Vijay Kumar, Marcia McNutt, Robert Merrifield, Brad Nelson, Brian Scassellati, Mariarosaria Taddeo, Russell Taylor, Manuela Veloso, Zhong Wang, Robert Wood, The grand challenges of science robotics, Sci. Robot. 3 (2018) eaar7650.

[2] Guanhao Xie, Duo Zhao, Qichao Tang, Muhua Zhang, Wenjie Zhao, Yewen Wang, Path planning for robotic arm based on reinforcement learning under the train, in: 2023 IEEE International Conference on Robotics and Biomimetics, ROBIO, 2023, pp. 1–8.

[3] Andrea Maria Zanchettin, Nicola Maria Ceriani, Paolo Rocco, Hao Ding, Björn Matthias, Safety in human-robot collaborative manufacturing environments: Metrics and control, IEEE Trans. Autom. Sci. Eng. 13 (2) (2016) 882–893.

[4] S. Ma, M. Konno, An obstacle avoidance scheme for hyper-redundant manipulators-global motion planning in posture space, in: Proceedings of International Conference on Robotics and Automation, Vol. 1, 1997, pp. 161–166.

[5] Farhad Aghili, A prediction and motion-planning scheme for visually guided robotic capturing of free-floating tumbling objects with uncertain dynamics, IEEE Trans. Robot. 28 (3) (2012) 634–649.

[6] Andreas Orthey, Constantinos Chamzas, Lydia E. Kavraki, Sampling-based motion planning: A comparative review, Annu. Rev. Control. Robot. Auton. Syst. (Volume 7, 2024) (2024) 285–310.

[7] Tobia Marcucci, Mark Petersen, David Wrangel, Russ Tedrake, Motion planning around obstacles with convex optimization, Sci. Robot. 8 (2023) eadf7843.

[8] Nathan Ratliff, Matt Zucker, J. Andrew Bagnell, Siddhartha Srinivasa, CHOMP: Gradient optimization techniques for efficient motion planning, in: 2009 IEEE International Conference on Robotics and Automation, 2009, pp. 489–494.

[9] John Schulman, Jonathan Ho, Alex X. Lee, Ibrahim Awwal, Henry Bradlow, P. Abbeel, Finding locally optimal, collision-free trajectories with sequential convex optimization, Robot.: Sci. Syst. IX (2013).

[10] Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, Stefan Schaal, STOMP: Stochastic trajectory optimization for motion planning, in: 2011 IEEE International Conference on Robotics and Automation, 2011, pp. 4569–4574.

[11] Giovanni Buizza Avanzini, Andrea Maria Zanchettin, Paolo Rocco, Constrained model predictive control for mobile robotic manipulators, Robotica 36 (2017) 1–20.

[12] Massimo Cefalo, Emanuele Magrini, Giuseppe Oriolo, Sensor-based task-constrained motion planning using model predictive control, IFAC-PapersOnLine 51 (2018) 220–225.

[13] Yunan Wang, Chuxiong Hu, Zeyang Li, Shize Lin, Suqin He, Yu Zhu, Time-optimal control for high-order chain-of-integrators systems with full state constraints and arbitrary terminal states, IEEE Trans. Autom. Control 70 (3) (2025) 1499–1514.

[14] Shize Lin, Chuxiong Hu, Suqin He, Wenxiang Zhao, Ze Wang, Yu Zhu, Real-time local greedy search for multiaxis globally time-optimal trajectory, IEEE Trans. Syst. Man Cybern.: Syst. 54 (2) (2024) 960–971.

[15] John T. Betts, Practical Methods for Optimal Control and Estimation Using Nonlinear Programming, Second Edition, second ed., Society for Industrial and Applied Mathematics, 2010.

[16] S. Boyd, L. Vandenberghe, L. Faybusovich, Convex optimization, IEEE Trans. Autom. Control 51 (11) (2006) 1859.

[17] Peter E. Hart, Nils J. Nilsson, Bertram Raphael, A formal basis for the heuristic determination of minimum cost paths, IEEE Trans. Syst. Sci. Cybern. 4 (2) (1968) 100–107.

[18] L.E. Kavraki, P. Svestka, J.-C. Latombe, M.H. Overmars, Probabilistic roadmaps for path planning in high-dimensional configuration spaces, IEEE Trans. Robot. Autom. 12 (4) (1996) 566–580.

[19] Steven M. LaValle, Rapidly-exploring random trees : a new tool for path planning, Annu. Res. Rep. (1998).

[20] J.J. Kuffner, S.M. LaValle, RRT-connect: An efficient approach to single-query path planning, in: Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065), Vol. 2, 2000, pp. 995–1001 vol.2.

[21] Oktay Arslan, Karl Berntorp, Panagiotis Tsiotras, Sampling-based algorithms for optimal motion planning using closed-loop prediction, in: 2017 IEEE International Conference on Robotics and Automation, ICRA, 2017, pp. 4991–4996.

[22] Ioan A. Şucan, Lydia E. Kavraki, Kinodynamic motion planning by interior-exterior cell exploration, in: Algorithmic Foundation of Robotics VIII: Selected Contributions of the Eight International Workshop on the Algorithmic Foundations of Robotics, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 449–464.

[23] Fahad Islam, Jauwairia Nasir, Usman Malik, Yasar Ayaz, Osman Hasan, RRT*-smart: Rapid convergence implementation of RRT* towards optimal solution, in: 2012 IEEE International Conference on Mechatronics and Automation, 2012, pp. 1651–1656.

[24] Oktay Arslan, Panagiotis Tsiotras, Use of relaxation methods in sampling-based algorithms for optimal motion planning, in: 2013 IEEE International Conference on Robotics and Automation, 2013, pp. 2421–2428.

[25] Jonathan D. Gammell, Siddhartha S. Srinivasa, Timothy D. Barfoot, Informed rRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic, in: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2014, pp. 2997–3004.

[26] A. Yershova, L. Jaillet, T. Simeon, S.M. LaValle, Dynamic-domain RRTs: Efficient exploration by controlling the sampling domain, in: Proceedings of the 2005 IEEE International Conference on Robotics and Automation, 2005, pp. 3856–3861.

[27] O. Khatib, Real-time obstacle avoidance for manipulators and mobile robots, in: Proceedings. 1985 IEEE International Conference on Robotics and Automation, Vol. 2, 1985, pp. 500–505.

[28] Fabrizio Flacco, Torsten Kröger, Alessandro De Luca, Oussama Khatib, A depth space approach to human-robot collision avoidance, in: 2012 IEEE International Conference on Robotics and Automation, 2012, pp. 338–345.

[29] Kelly Merckaert, Bryan Convens, Chi ju Wu, Alessandro Roncone, Marco M. Nicotra, Bram Vanderborght, Real-time motion control of robotic manipulators for safe human–robot coexistence, Robot. Comput.-Integr. Manuf. 73 (2022) 102223.

[30] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, Paulo Tabuada, Control barrier functions: Theory and applications, in: 2019 18th European Control Conference, ECC, 2019, pp. 3420–3431.

[31] Federica Ferraguti, Chiara Talignani Landi, Silvia Costi, Marcello Bonfé, Saverio Farsoni, Cristian Secchi, Cesare Fantuzzi, Safety barrier functions and multi-camera tracking for human-robot shared environment, Robot. Auton. Syst. 124 (2019) 103388.

[32] Hugo Nascimento, Martin Mujica, Mourad Benoussaad, Collision avoidance interaction between human and a hidden robot based on kinect and robot data fusion, IEEE Robot. Autom. Lett. 6 (1) (2021) 88–94.

[33] Matteo Parigi Polverini, Andrea Maria Zanchettin, Paolo Rocco, A computationally efficient safety assessment for collaborative robotics applications, Robot. Comput.-Integr. Manuf. 46 (2017) 25–37.

[34] Kelly Merckaert, Bryan Convens, Marco Nicotra, Bram Vanderborght, Real-time constraint-based planning and control of robotic manipulators for safe human–robot collaboration, Robot. Comput.-Integr. Manuf. 87 (2024) 102711.

[35] Hossein Abdi, Golnaz Raja, Reza Ghabcheloo, Safe control using vision-based control barrier function (V-CBF), in: 2023 IEEE International Conference on Robotics and Automation, ICRA, 2023, pp. 782–788.

[36] Suqin He, Weiye Zhao, Chuxiong Hu, Yu Zhu, Changliu Liu, A hierarchical long short term safety framework for efficient robot manipulation under uncertainty, Robot. Comput.-Integr. Manuf. 82 (2023) 102522.

[37] Jichuan Yu, Zhao Jin, Chuxiong Hu, Jizhou Yan, Ze Wang, Yu Zhu, Hierarchical real-time motion planning for safe multi-robot manipulation in dynamic environments*, in: 2024 International Conference on Advanced Robotics and Mechatronics, ICARM, 2024, pp. 113–119.

[38] Vidyasagar Rajendran, Pamela Carreno-Medrano, Wesley Fisher, Dana Kulić, Human-aware RRT-connect: Motion planning for safe human-robot collaboration, in: 2021 30th IEEE International Conference on Robot & Human Interactive Communication (RO-MAN), 2021, pp. 15–22.

[39] Lucas Joseph, Joshua K. Pickard, Vincent Padois, David Daney, Online velocity constraint adaptation for safe and efficient human-robot workspace sharing, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2020, pp. 11045–11051.

[40] Andrea Pupa, Mohammad Arrfou, Gildo Andreoni, Cristian Secchi, A safety-aware kinodynamic architecture for human-robot collaboration, IEEE Robot. Autom. Lett. 6 (3) (2021) 4465–4471.

[41] Jakob Thumm, Matthias Althoff, Provably safe deep reinforcement learning for robotic manipulation in human environments, in: 2022 International Conference on Robotics and Automation, ICRA, 2022, pp. 6344–6350.

[42] Hao-Lun Hsu, Qiuhua Huang, Sehoon Ha, Improving safety in deep reinforcement learning using unsupervised action planning, in: 2022 International Conference on Robotics and Automation, ICRA, 2022, pp. 5567–5573.

[43] Zeyang Li, Chuxiong Hu, Yunan Wang, Yujie Yang, Shengbo Eben Li, Safe reinforcement learning with dual robustness, IEEE Trans. Pattern Anal. Mach. Intell. 46 (12) (2024) 10876–10890.

[44] Martin Ester, Hans-Peter Kriegel, Jörg Sander, Xiaowei Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: Proceedings of the Thirteenth National Conference on Artificial Intelligence, KDD '96, AAAI Press, 1996, pp. 226–231.
[45] Scikit-learn Developers, Clustering, 2024, Accessed: December 2, 2024.
[46] Tuomas Haarnoja, Aurick Zhou, P. Abbeel, Sergey Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018, ArXiv, abs/1801.01290.
[47] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, G. Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, P. Abbeel, Sergey Levine, Soft actor-critic algorithms and applications, 2018, ArXiv, abs/1812.05905.
[48] Xin Wang, Yudong Chen, Wenwu Zhu, A survey on curriculum learning, IEEE Trans. Pattern Anal. Mach. Intell. 44 (9) (2022) 4555–4576.
[49] Qian-Yi Zhou, Jaesik Park, Vladlen Koltun, Open3D: A modern library for 3D data processing, 2018, arXiv:1801.09847.
[50] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, Noah Dormann, Stable-Baselines3: Reliable reinforcement learning implementations, J. Mach. Learn. Res. 22 (268) (2021) 1–8.
[51] Ioan A. Şucan, Mark Moll, Lydia E. Kavraki, The open motion planning library, IEEE Robot. Autom. Mag. 19 (4) (2012) 72–82, https://ompl.kavrakilab.org.